

软件工程

分析软件需求

刘 爽

中国人民大学

本课程部分教学内容基于：《软件工程-理论与实践》，毛新军、董威编著，高等教育出版社，2024.1，ISBN 978-7-04-061010-9 配套课件由出版社/作者提供，仅用于教学

内容

1. 分析软件需求概述

- ✓ 分析软件需求的任务

2. 结构化需求分析方法

- ✓ 过程、建模语言和需求模型

3. 面向对象需求分析方法

- ✓ 过程、建模语言和需求模型

4. 软件需求文档化及评审

- ✓ 软件需求规格说明书及评审



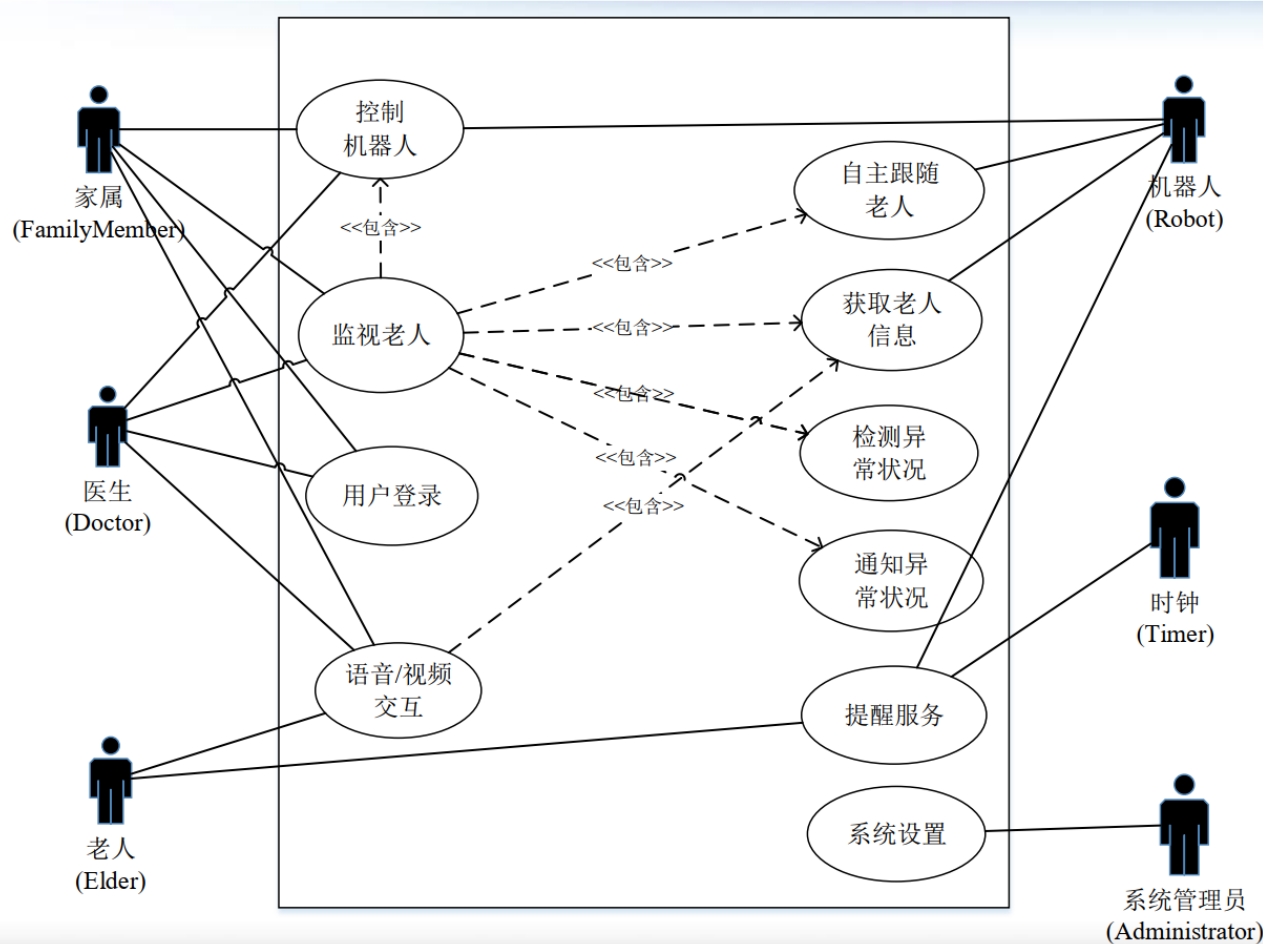
1.1 为什么要分析软件需求

□初步软件需求存在的问题

- ✓ **需求不详尽**，没有提供软件需求细节性内容，无法指导后续设计
- ✓ **描述不清晰**，对软件需求的描述仍然不够清晰、直观
- ✓ **关系不明朗**，未深入分析不同软件需求项之间的关系
- ✓ **存在潜在缺陷**，潜在的需求缺陷难以被发现
- ✓ **没有区分不同的软件需求**，有必要鉴别不同软件需求项的重要性差别，区分不同软件需求的开发优先级

软件需求还是“粗胚”，需要“打磨”

示例：初步软件需求描述中存在的问题(1/2)



这张图说清楚软件需求了吗？
还有哪些需求没有说清楚？

示例：初步软件需求描述中存在的问题(2/2)

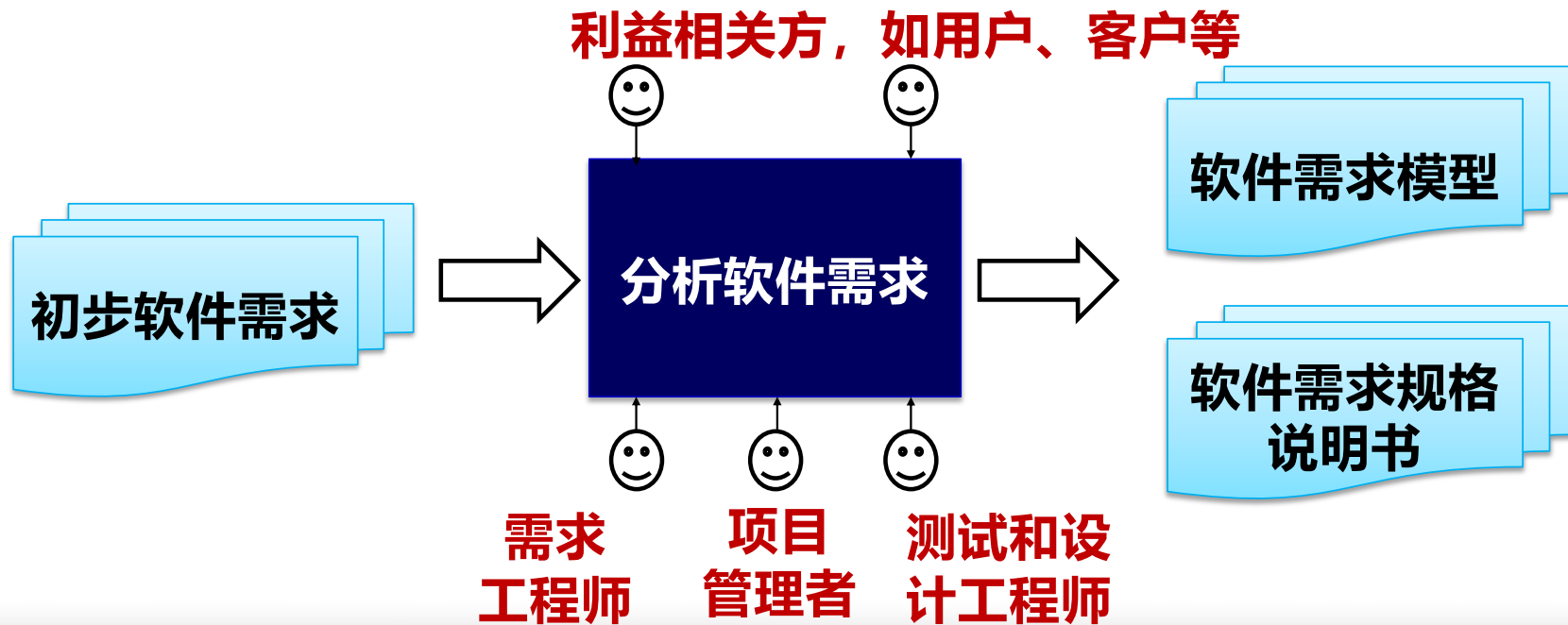
□ “退票” 的功能描述

- ✓ 用户首先登录到系统中，选择要退票的车票，系统将自动地完成退票工作，并提示用户退票的结果

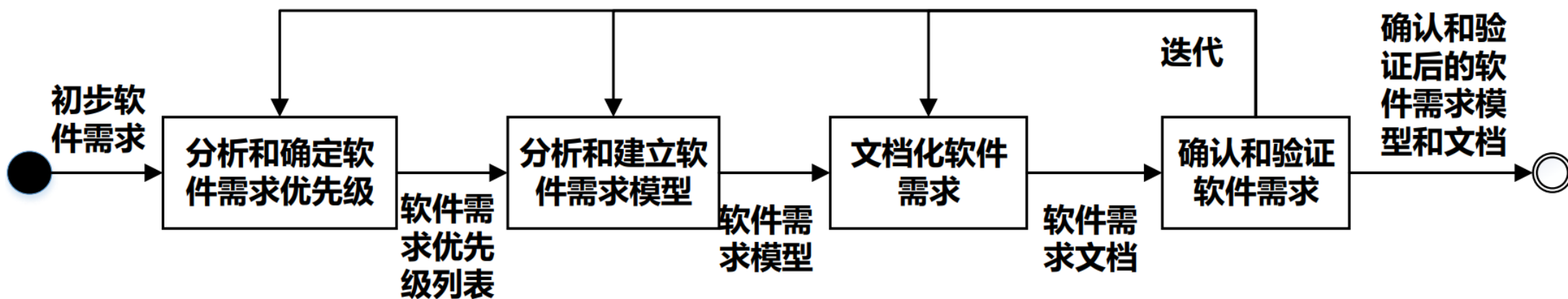
这段需求描述将“退票”的需求讲清楚了吗？为什么？

1.2 分析软件需求的任务

□ 基于初步软件需求，进一步**精化和分析**软件需求，确定软件需求**优先级**，建立软件**需求模型**，发现和解决软件需求**缺陷**，形成高质量的软件需求模型和软件**需求规格说明书**



需求分析过程



内容

1. 分析软件需求概述

✓分析软件需求的任务

2. 结构化需求分析方法

✓过程、建模语言和需求模型

3. 面向对象需求分析方法

✓过程、建模语言和需求模型

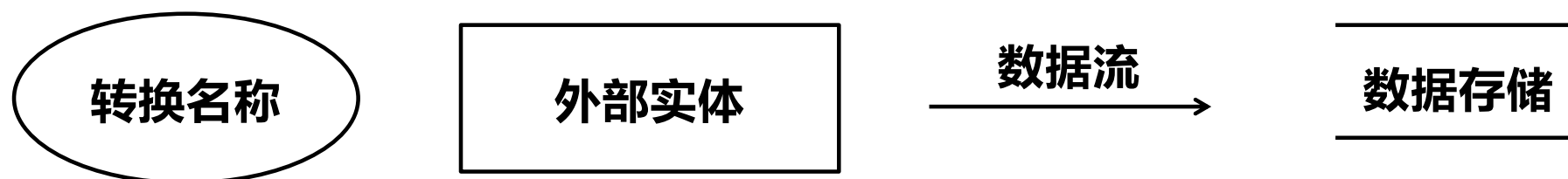
4. 软件需求文档化及评审

✓软件需求规格说明书及评审

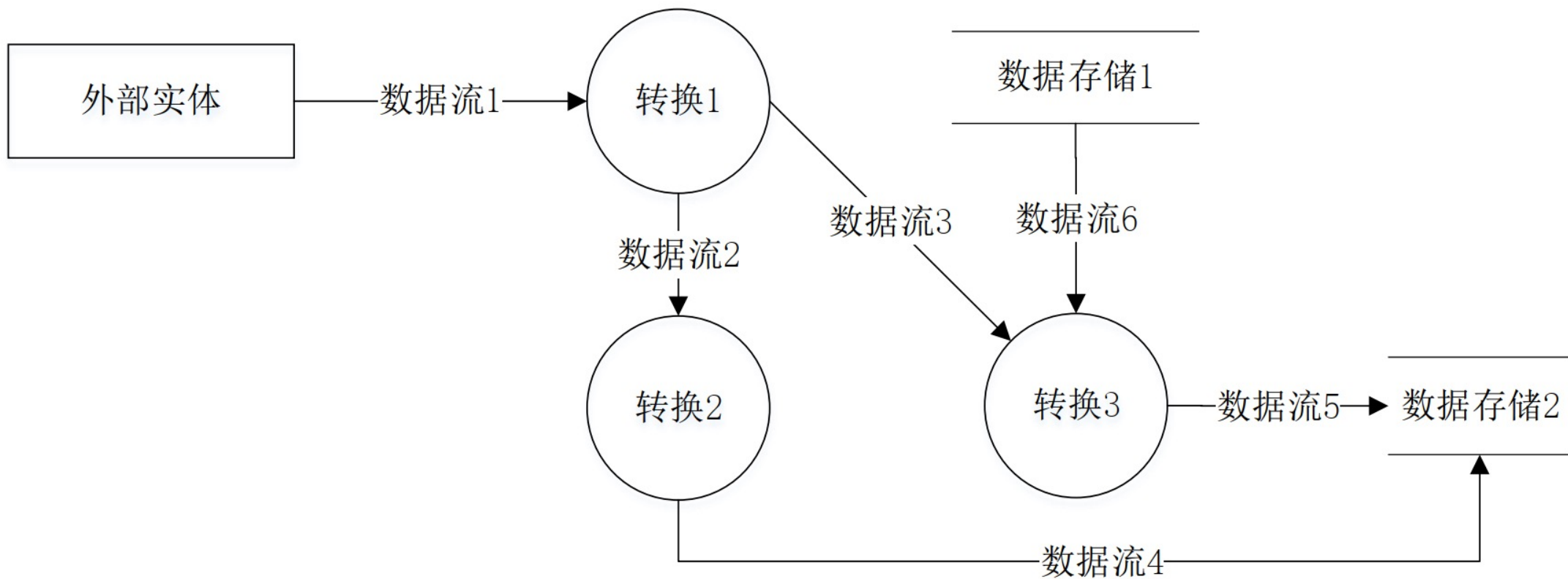


2.1 基本思想

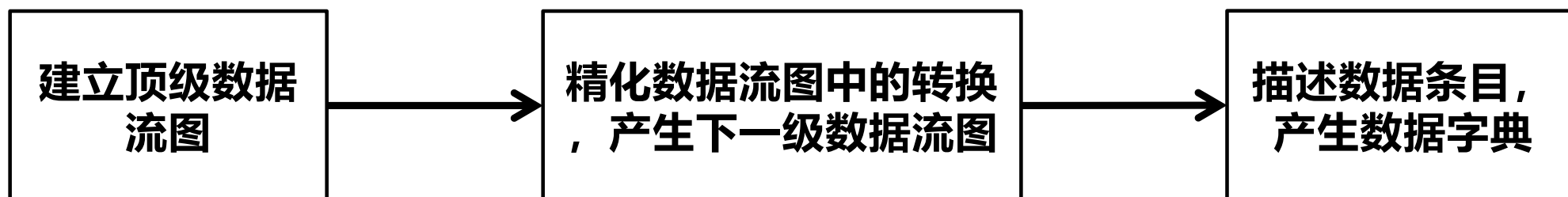
- 功能体现为**对数据的加工和处理**
- 绘制**数据流图**来描述软件的功能
- 通过**逐步分解**来细化软件的功能



数据流图示意图



2.2 面向数据流的需求分析过程



步骤1：建立顶级数据流图

□ 顶级数据流图用于描述软件系统与其外部环境间的关系

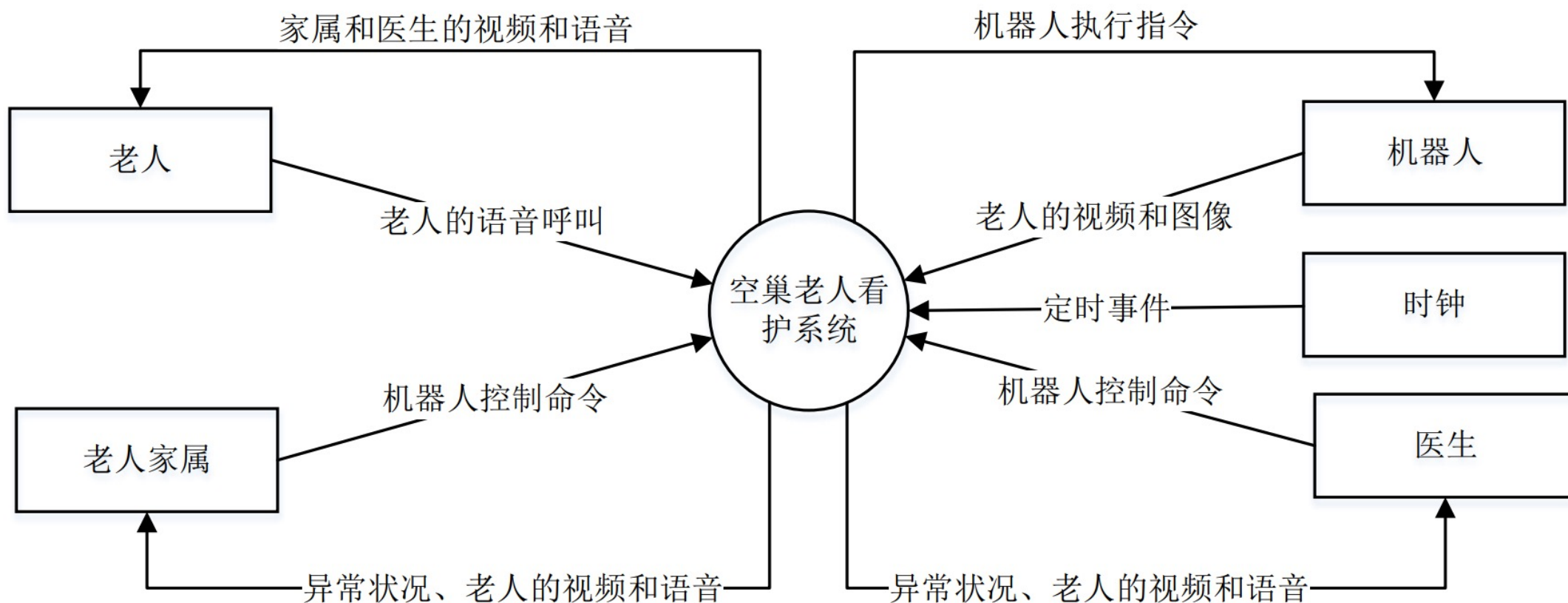
- ✓ 外部环境有哪些**实体** - **外部实体**
- ✓ 它们与软件系统要进行什么样的**交互** - **数据流**

□ 顶级数据流图的组成

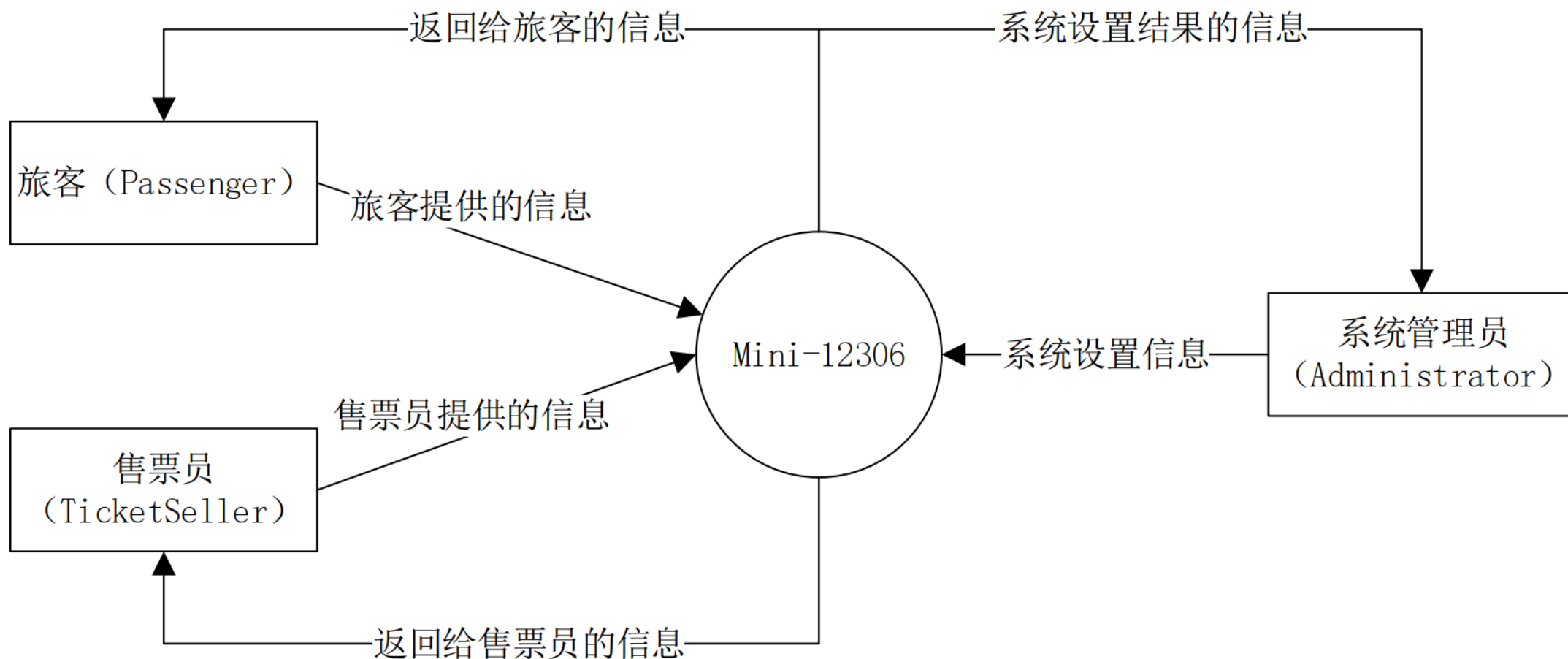
- ✓ 将待开发的软件系统抽象和表示为一个转换，仅仅关注和刻画软件与其环境之间的**交互**
- ✓ 只有一个**转换**，对应于待开发的软件系统

□ 也称为0级数据流图

示例：“空巢老人看护软件”的顶级数据流图



示例：“Mini-12306”的顶级数据流图

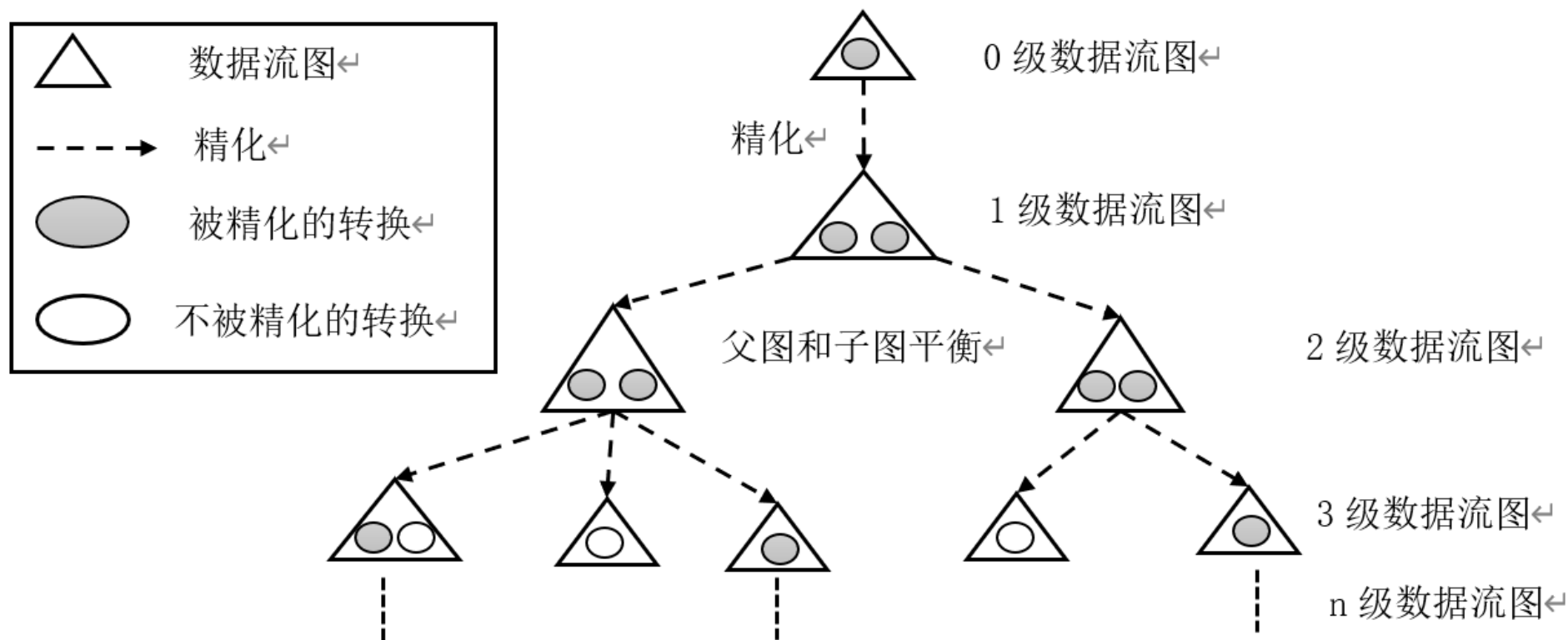


步骤2：精化数据流图中的转换

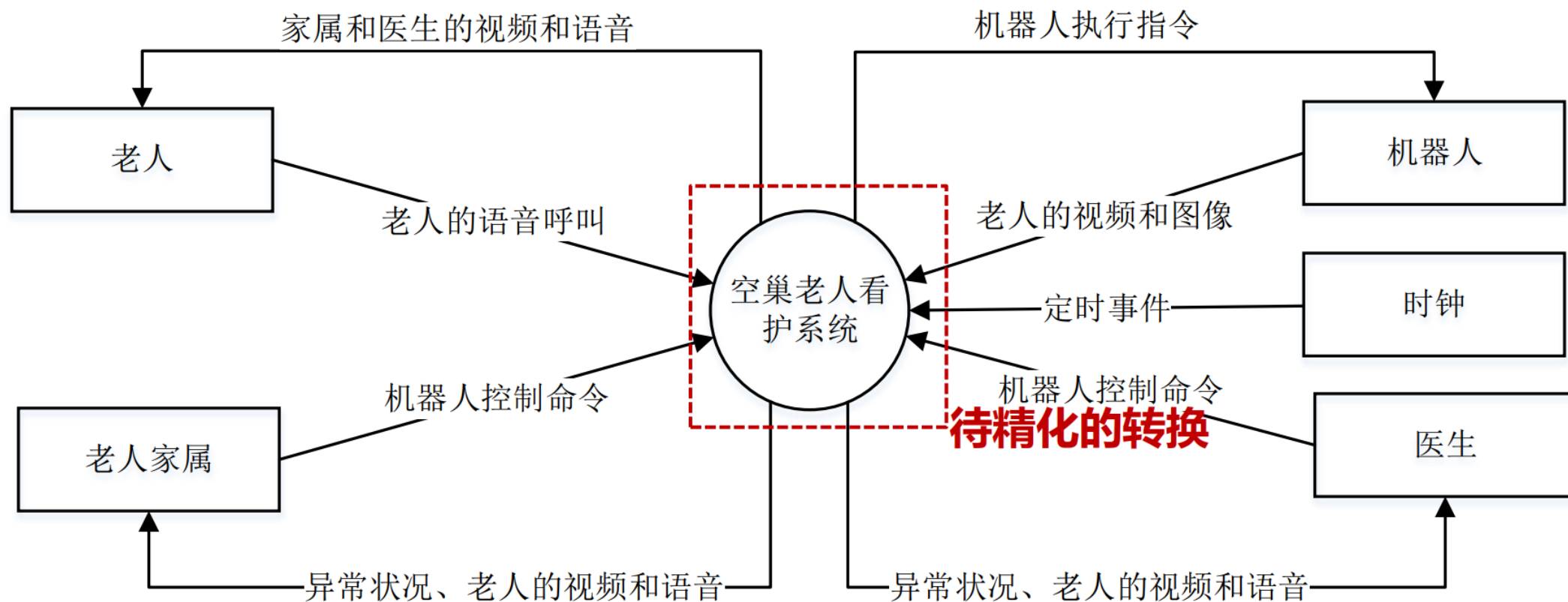
- 采用**自顶向下、逐步求精**策略对数据流图进行精化和分析
 - ✓基于用户的**文字性需求描述**，逐个分析数据流图中的转换
 - ✓如果某个转换的功能粒度大，那么就需要对其进行**精化**，产生针对该转换的**下一级数据流图**
 - ✓形成一组**层次化的数据流图**，它们共同构成了基于数据流的软件需求模型

将粗粒度的“转换”精化为“数据流图”

对数据流图总“转换”进行精化的示意图

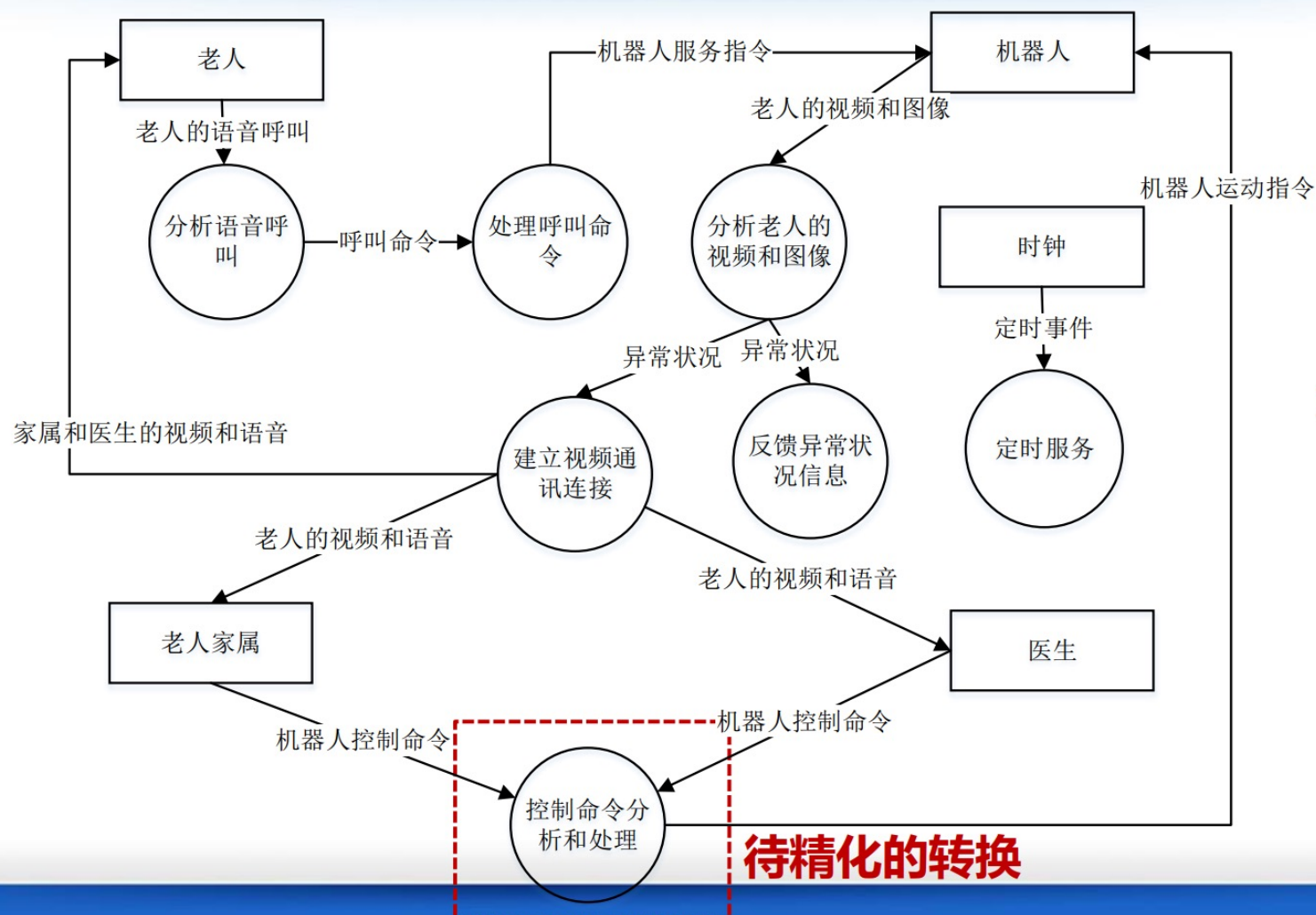


示例：“空巢老人看护软件” 转换精化(1/3)



0级数据流图

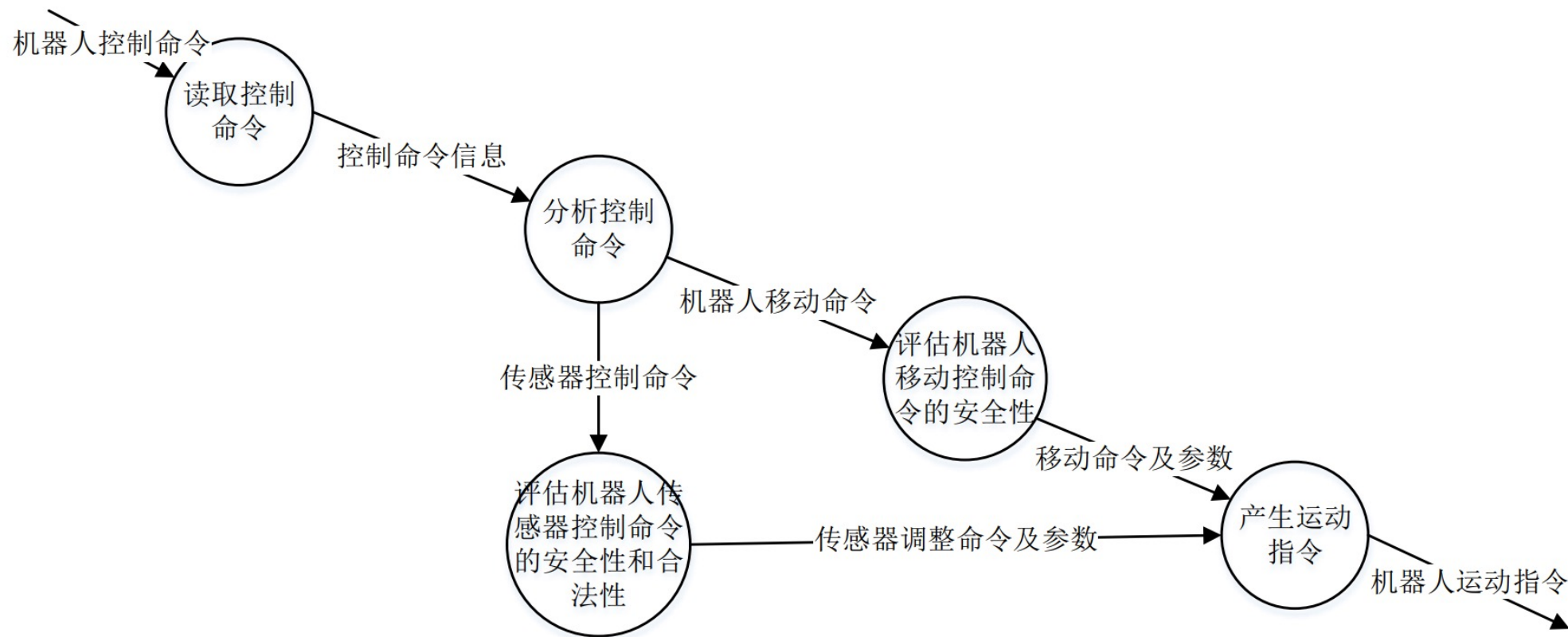
示例：“空巢老人看护软件” 转换精化(2/3)



1级数据流图

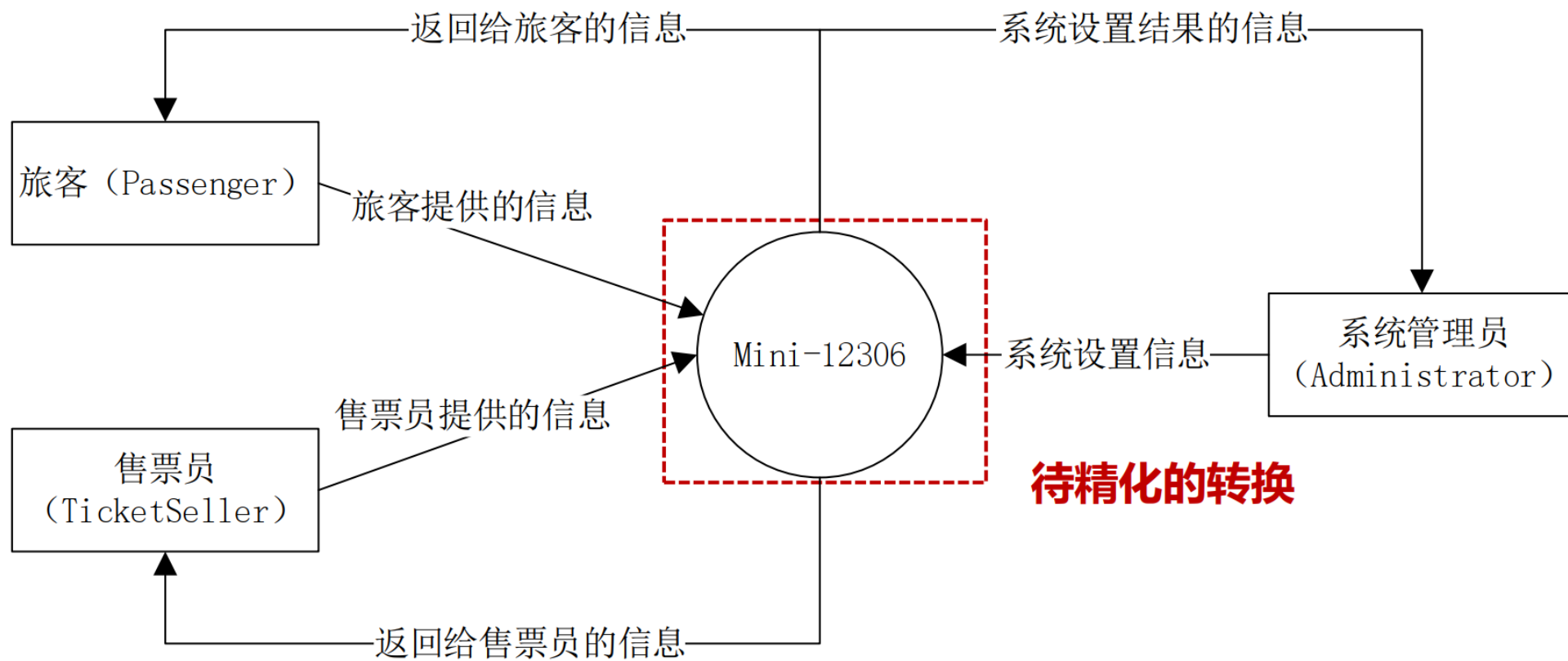
待精化的转换

示例：“空巢老人看护软件” 转换精化(3/3)



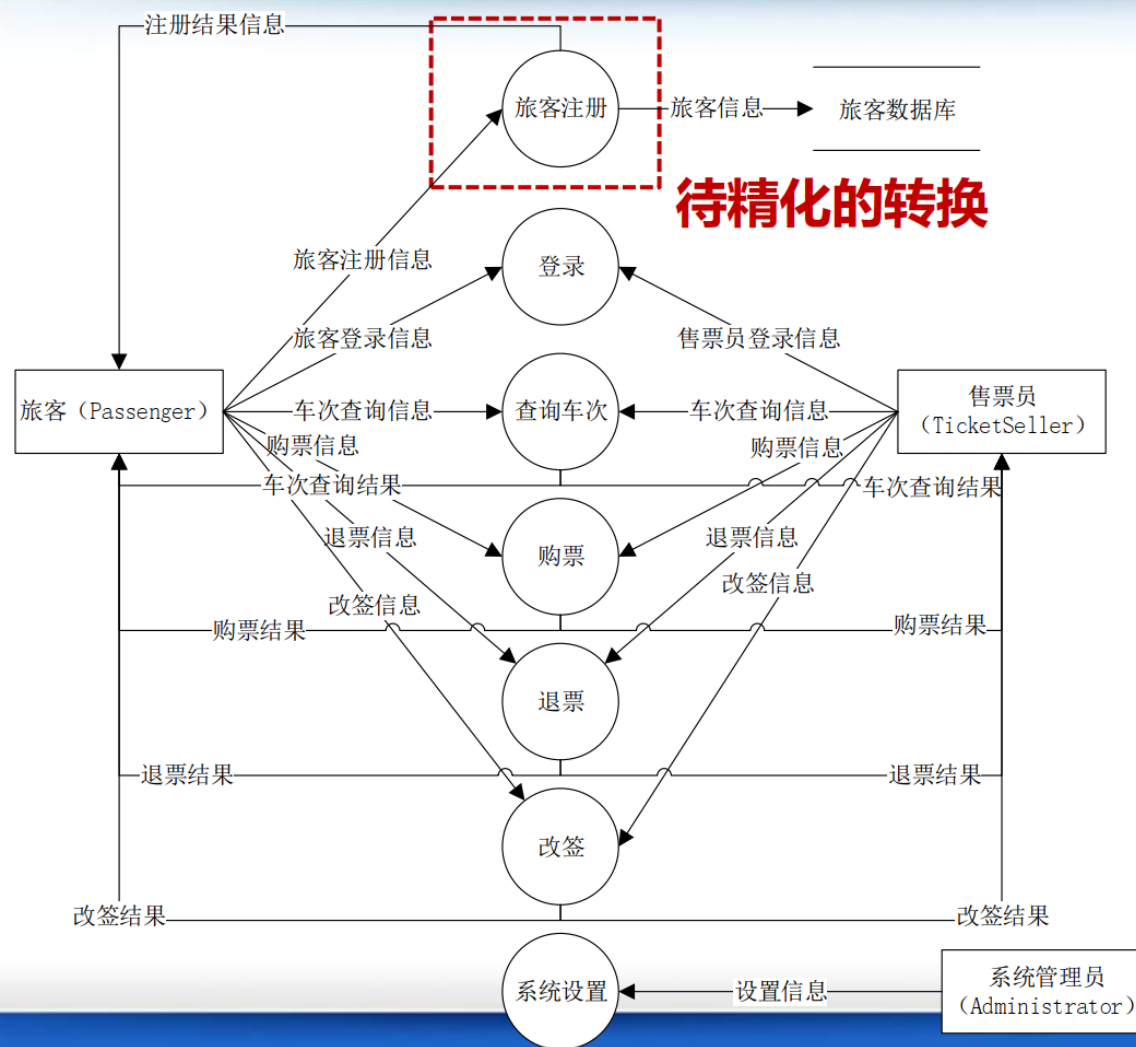
2级数据流图

示例：“Mini-12306” 转换精化(1/3)

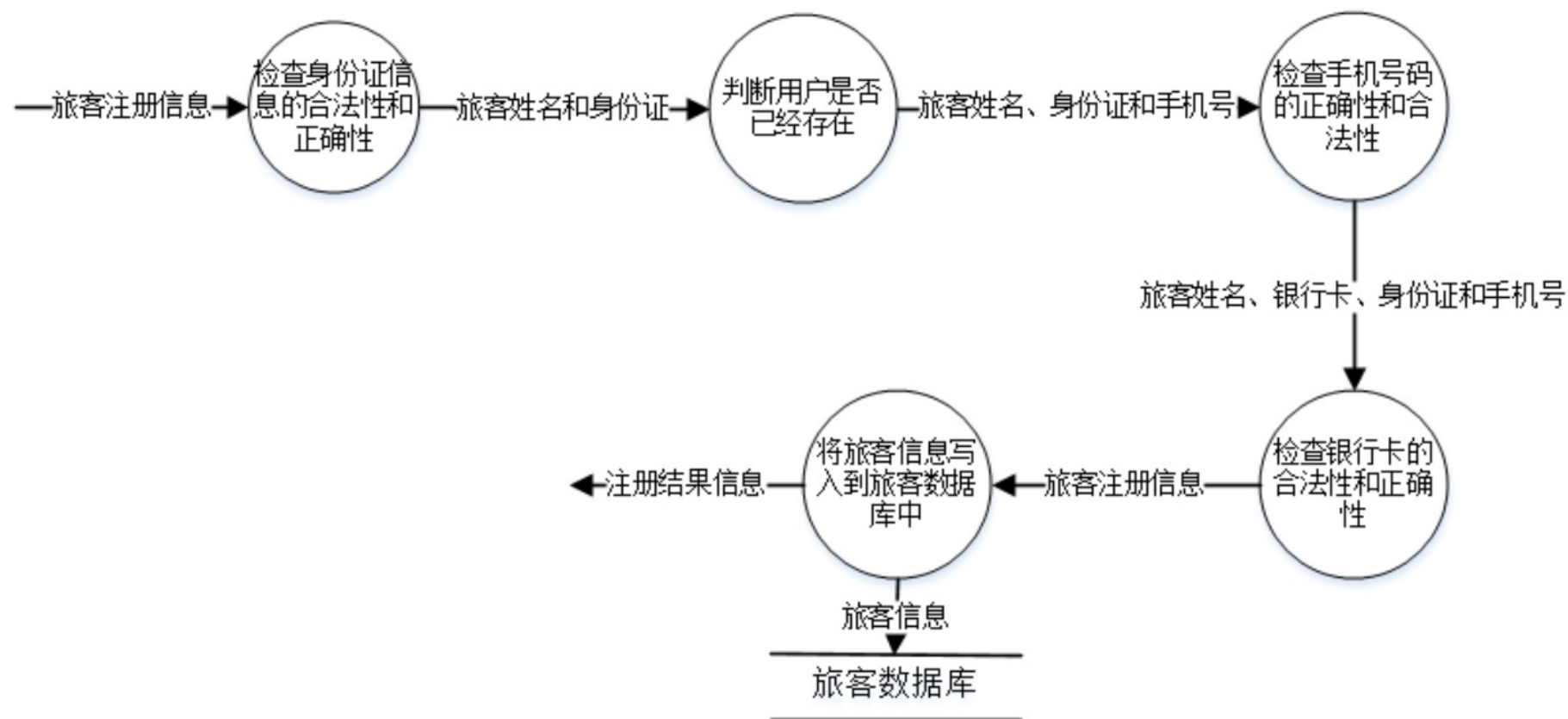


0级数据流图

示例：“Mini-12306” 转换精化(2/3)



示例：“Mini-12306” 转换精化(3/3)



2级数据流图

精化过程中确保父图与子图的平衡

- 针对父图中被精化的转换，它的**每一个输入和输出数据流要与精化后所产生的数据流图中的输入或输出数据流保持一致**
- 这一策略确保精化过程中不会漏掉待精化转换的所有输入和输出数据流，也不会~~在精化所产生的数据流图中平白无故地增加外部的输入或输出数据流。~~

步骤3：建立数据字典

- 数据流图并没有详尽地说清楚转换、数据流等的**具体细节**
- 面向数据流的需求分析方法学还提供了**数据字典**
 - ✓字典中的每一个条目可用于详细描述各个数据流、外部实体、数据存储、不再被精化的转换，进而获得关于这些建模元素的详细信息

示例：“机器人移动命令”的数据条目描述

移动命令 = 向前移动命令 + 向后移动命令 + 左转命令 + 右转命令	
向前移动命令	MovingForward + StepLength
向后移动命令	MovingBackward + StepLength
左转命令	TurnLeft + TurningDegree (default = 30 degree)
右转命令	TurnRight + TurningDegree (default = 30 degree)
StepLength	表示移动距离，类型为实数，缺省是30厘米
TurningDegree	表示转向角度，类型为整数，缺省是30度

示例：“旅客注册信息”的数据条目

旅客注册信息 = 旅客姓名 + 身份证 + 银行卡 + 手机号

旅客姓名	长度为5的汉字字符串
-------------	------------

身份证	18位的身份证号码
------------	-----------

银行卡	开户行 + 银行卡号
------------	------------

手机号	11位的数字串
------------	---------

2.3 注意事项

- 数据流、外部实体、数据存储要用**名词或名词短语**来描述
- 转换要用**动词或动名词**来描述
- 确保父图和子图的**平衡**
- 精化**适可而止**

内容

1. 分析软件需求概述

✓分析软件需求的任务

2. 结构化需求分析方法

✓过程、建模语言和需求模型

3. 面向对象需求分析方法

✓过程、建模语言和需求模型

4. 软件需求文档化及评审

✓软件需求规格说明书及评审



3.1 软件需求的不同视角表示

□用例视角

- ✓ 具有哪些**功能**、功能间有何**关系**、功能与利益相关方有何**关系**
- ✓ UML提供了**用例图**来分析和描述用例视角的软件需求模型

□行为视角

- ✓ 用例是如何通过业务领域中一组对象以及它们间的**交互**来达成的
- ✓ UML提供了**交互图**、**状态图**来描述行为视角的软件需求模型

□结构视角

- ✓ 业务领域有哪些重要的领域**概念**以及它们之间具有什么样的**关系**
- ✓ UML提供了**类图**来描述和分析业务领域的概念模型

需求的功能、行为和结构视点有何本质区别？

支持需求建模和分析的UML图

需求模型	视角	软件需求的内涵	需求阶段
用例图	用例	软件的功能及相互间的关系	获取软件需求
交互图	行为	功能完成所涉及的对象及相互间的交互和协作	分析软件需求
状态图	行为	对象的状态变化	分析软件需求
类图	结构	业务领域概念及相互关系	分析软件需求

顺序图

视点	图 (diagram)	说明
结构	包图 (package diagram)	从包层面描述系统的静态结构
	类图 (class diagram)	从类层面描述系统的静态结构
	对象图 (object diagram)	从对象层面描述系统的静态结构
	构件图(component diagram)	描述系统中构件及其依赖关系
行为	状态图(statechart diagram)	描述状态的变迁
	活动图(activity diagram)	描述系统活动的实施
	通信图(communication diagram)	描述对象间的消息传递与协作
	顺序图(sequence diagram)	描述对象间的消息传递与协作
部署	部署图 (deployment diagram)	描述系统中工件在物理运行环境中的部署情况
用例	用例图 (use case diagram)	从外部用户角度描述系统功能

3.1.1 UML交互图的作用

□刻画对象间的消息传递，分析如何通过交互协作完成功能

- ✓用例的功能实现方式
- ✓软件系统在某种使用场景下对象间的交互协作流程
- ✓软件系统的某个复杂操作的逻辑实现模型

□二类交互图

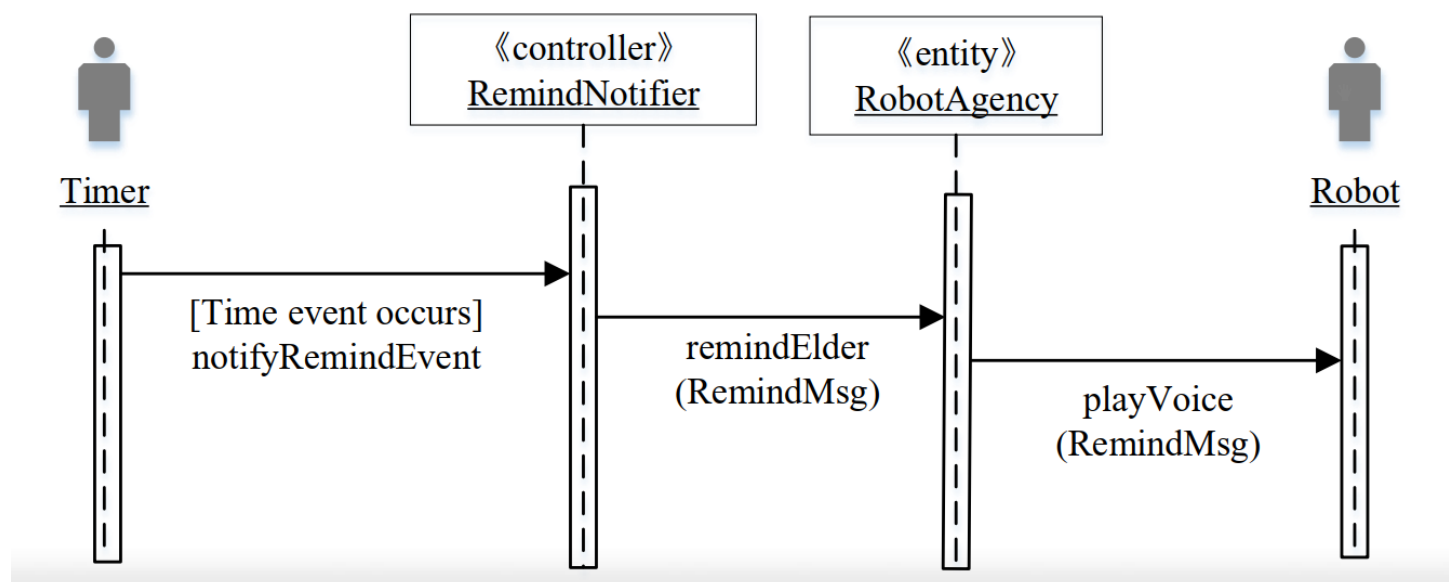
- ✓顺序图(Sequence Diagram)：强调消息传递的时间序
- ✓通信图(Communication Diagram)：突出对象间的合作

□表达能力相同，二类图可相互转换

顺序图

□描述对象间的消息交互序列

- ✓纵向：时间轴，对象及其生命线(虚线)和活跃期(长条矩形)
- ✓横向：对象间的消息传递



顺序图的表示方式

□对象

- ✓ “[对象名] : [类名]”
- ✓ 示例: “Tom: Student”或“Student”

□消息传递

- ✓ 对象生命线间的**有向边**
- ✓ “[*][监护条件] [返回值:=]消息名[(参数表)]”
- ✓ “*”为迭代标记表示同一消息对同一类的多个对象发送

对象间的消息传递

□同步消息

- ✓发送者等待接收者将消息处理完后再继续

□异步消息

- ✓发送者在发送完消息后不等待接收方即继续自己的处理

□自消息

- ✓一个对象发送给自身的消息

□返回消息

- ✓某条消息处理已经完成，处理结果沿返回消息传回

□创建消息和销毁消息

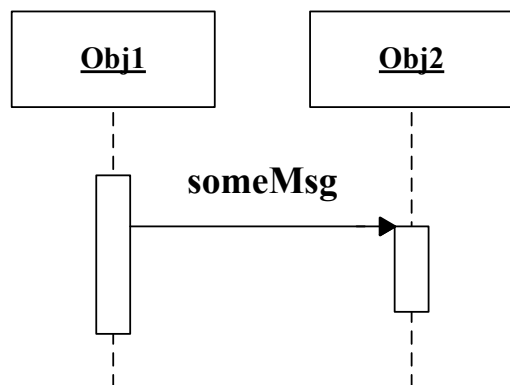
- ✓消息传递目标对象的创建和删除

消息图元的表示

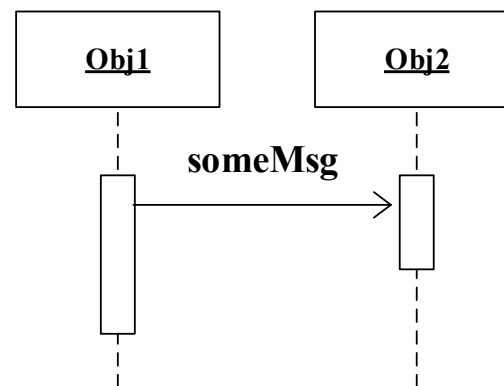
注意箭头的图形表示(线条和箭头形状)

注意图元符号!

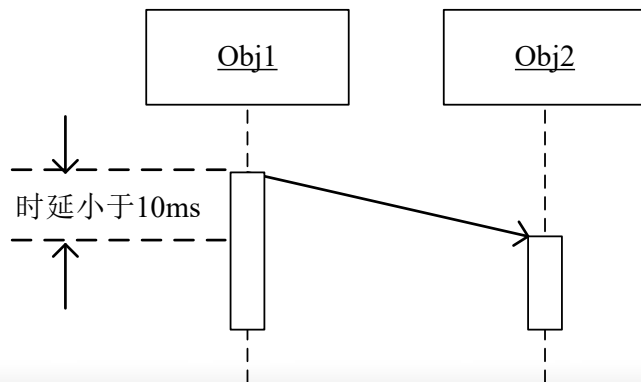
同步消息



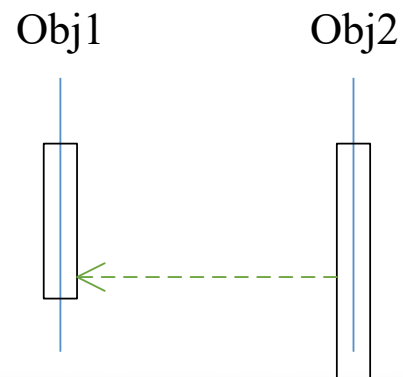
异步消息



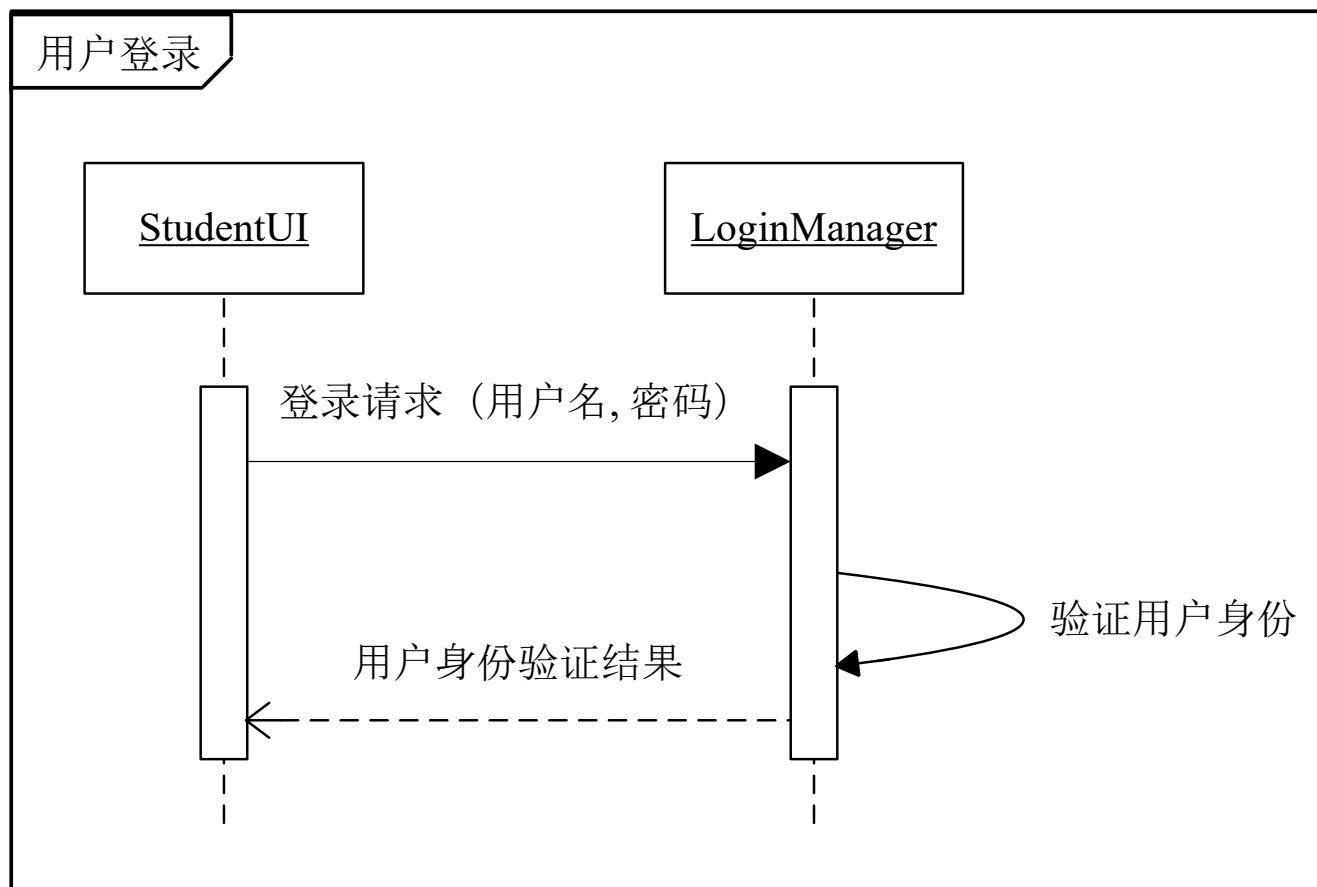
带时间
延迟的
消息



返回消息



示例：顺序图



- ✓ 对象
- ✓ 生命线
- ✓ 活跃期
- ✓ 同步消息
- ✓ 返回消息
- ✓ 自消息
- ✓ 消息名称
- ✓ 对象名称

特别注意图符的画法和要求

通信图的表示

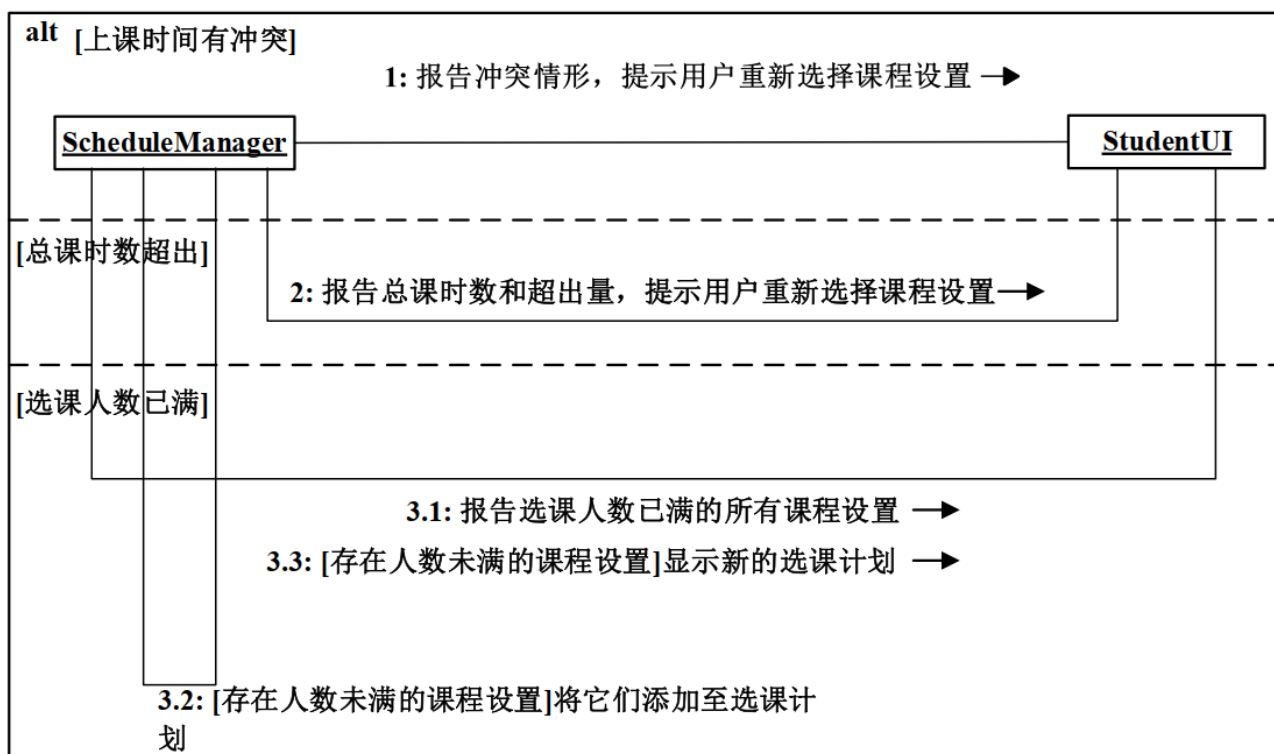
□节点表示对象

□对象间连接称为连接器

□连接器上可标示一到多条消息

□消息传递方向用靠近消息小箭头表示

□消息序号采用多层标号



顺序图和通信图的选取原则

□顺序图和通信图**语义上等价**

- ✓没必要针对同一建模目标同时创建这二个图

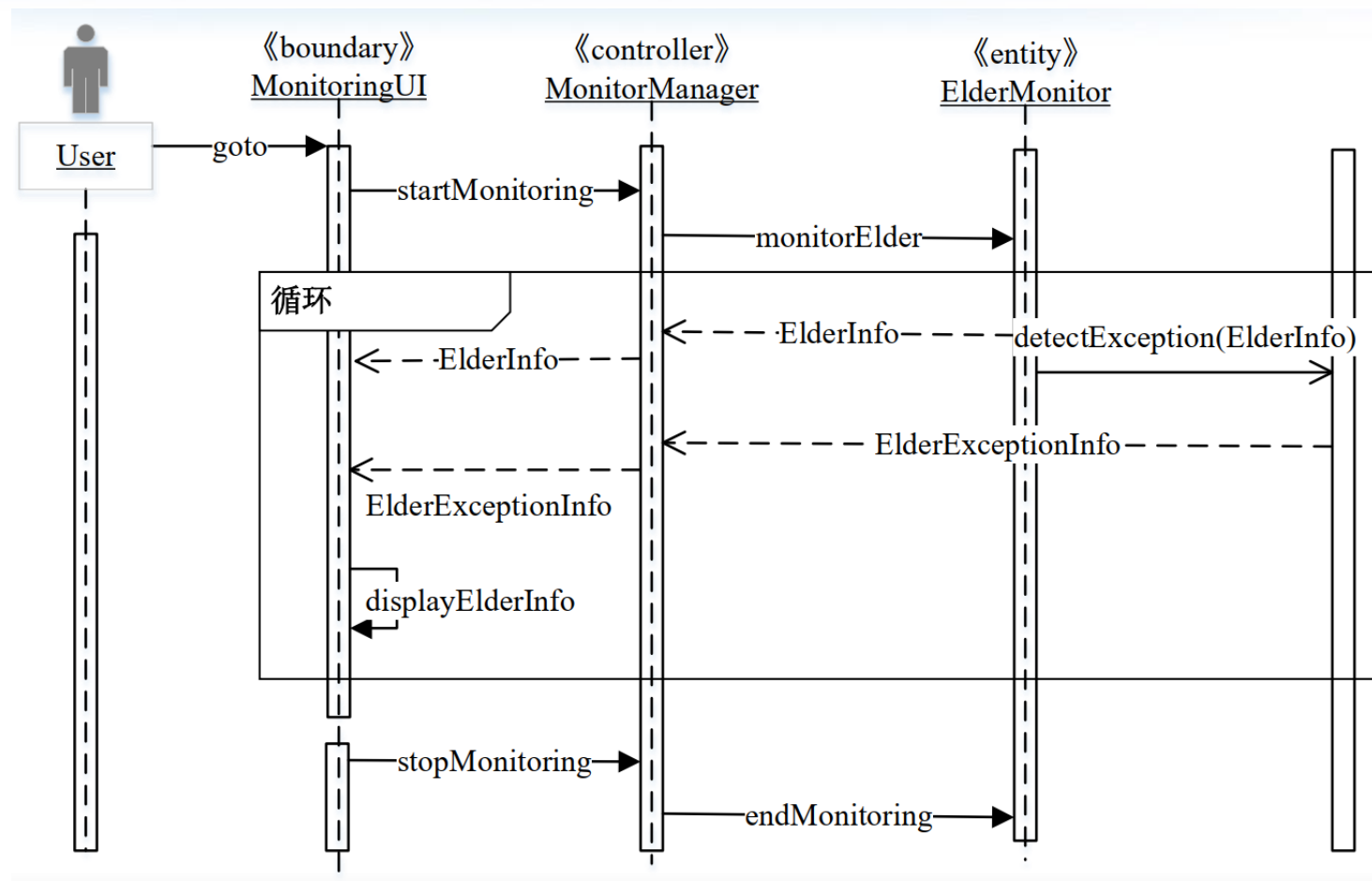
□选择原则

- ✓当需要强调消息传递的**时间序**时采用顺序图，当需要强调对象间的**交互协作关系**时采用通信图
- ✓当刻画场景或用例的动作序列时采用顺序图，当刻画软件内部某项功能的实现构想时，采用通信图
- ✓在业务分析和需求建模阶段优先考虑顺序图；在设计和实现阶段优先考虑通信图

示例：顺序图

□指出顺序图中各个图形要素及其含义

□这个图是否有画的不之处



类和对象图

视点	图 (diagram)	说明
结构	包图 (package diagram)	从包层面描述系统的静态结构
	类图 (class diagram)	从类层面描述系统的静态结构
	对象图 (object diagram)	从对象层面描述系统的静态结构
	构件图(component diagram)	描述系统中构件及其依赖关系
行为	状态图(statechart diagram)	描述状态的变迁
	活动图(activity diagram)	描述系统活动的实施
	通信图(communication diagram)	描述对象间的消息传递与协作
	顺序图(sequence diagram)	描述对象间的消息传递与协作
部署	部署图 (deployment diagram)	描述系统中工件在物理运行环境中的部署情况
用例	用例图 (use case diagram)	从外部用户角度描述系统功能

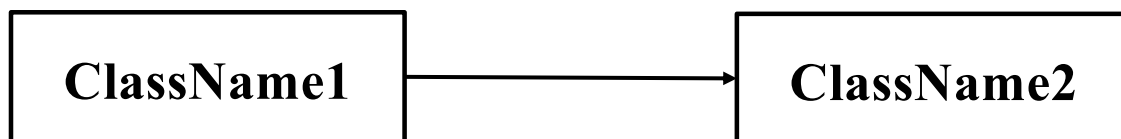
3.1.2 类图

□功效

- ✓描述系统的**类构成**，刻画系统的**静态组成结构**

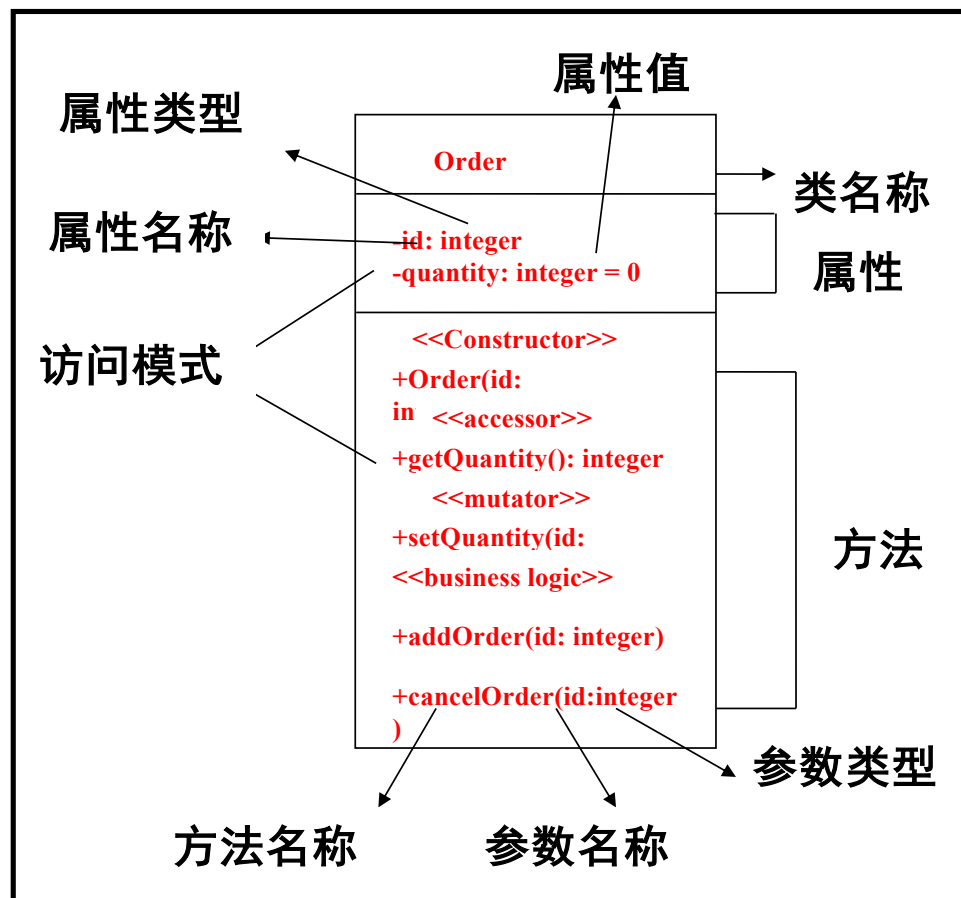
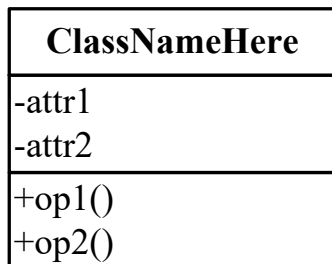
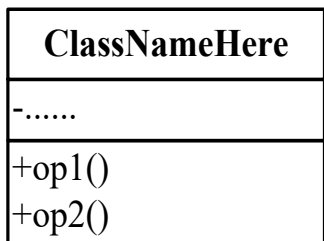
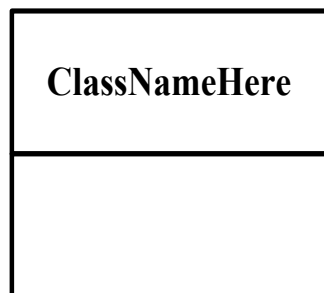
□图的构成

- ✓**结点**：表示系统中的类（或接口）及其属性和操作
- ✓**边**：类之间的关系



特别注意图符的画法和要求

类的UML表示



✓ 类名
✓ 属性
✓ 操作

特别注意图符的画法和要求

属性的表示

□[可见性] 名称 [: 类型] [多重性] [= 初值] [{约束特性}]

□可见性

- ✓ 公开(+): **所有对象**均可访问
- ✓ 保护(#): **所在类及子类对象**均可访问
- ✓ 私有(-): **仅所在类的对象**才可访问

□多重性: 属性取值数量, 如**1**, **0..1**, **0..***, **1..***, *****

□约束特性

- ✓ 可更改性: {readOnly}表示只读, 缺省为{changeable}
- ✓ 顺序性: {ordered}表示属性取值是有序的, 缺省为{unordered}
- ✓ 唯一性: {bag}表示属性取值元素允许出现重复元素 (缺省)
- ✓ 静态性: {static}表示静态属性, 属性值由类所有实例对象共享

方法的表示

□[可见性] 名称[(参数表)] [: 返回类型] [{约束特性}]

□操作的约束特性

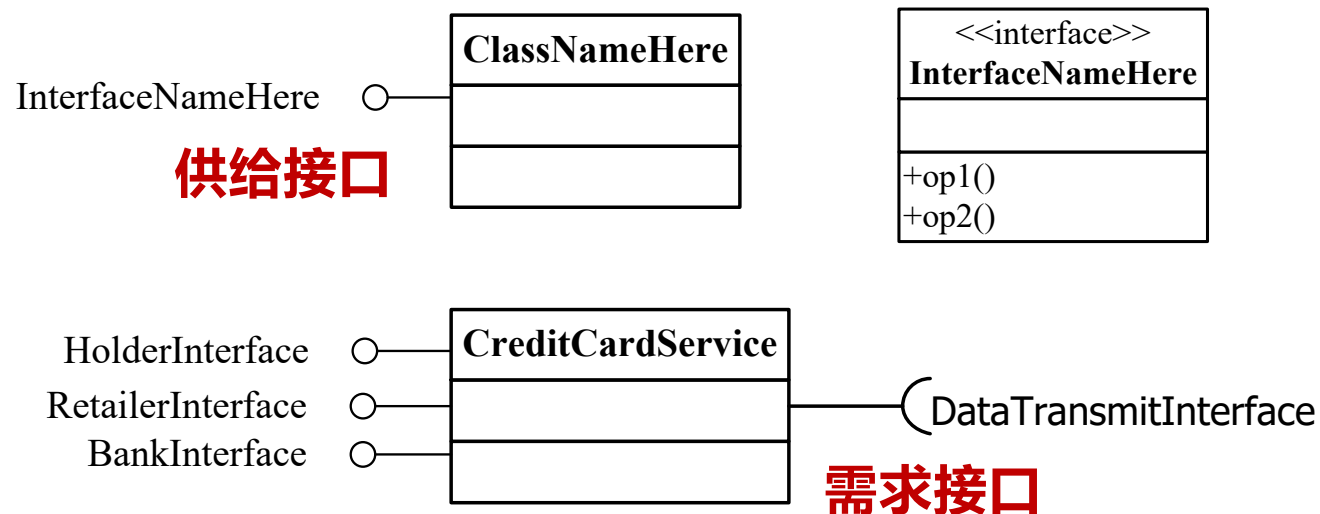
- ✓ **查询操作**: {isQuery = true}表示查询操作, { isQuery = false}表示修改操作, 缺省为修改操作。
- ✓ **多态性**: {isPolymorphic = true}表示本操作允许多态, 即可被子类中相同定义形式的操作所覆盖
- ✓ **并发性**: {concurrency = sequential} 任一时刻只有一个对象调用可执行。{concurrency = guarded} 并行线程可同时调用多个对象的本操作, 但同一时刻只允许一个调用执行。{concurrency = concurrent} 并行线程可以同时调用多个对象的本操作且这些调用可并发执行
- ✓ **异常**: 操作在执行过程中可能引发异常

接口(Interface)

□ 一种**不包含操作实现部分**的特殊类，接口的形式

✓ **供给接口**：对外提供的接口

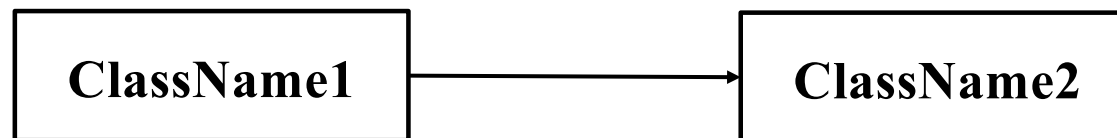
✓ **需求接口**：需要使用的接口



特别注意图符的画法和要求

类间的关系

- 关联
- 依赖
- 继承
- 实现
- 聚合
- 组合

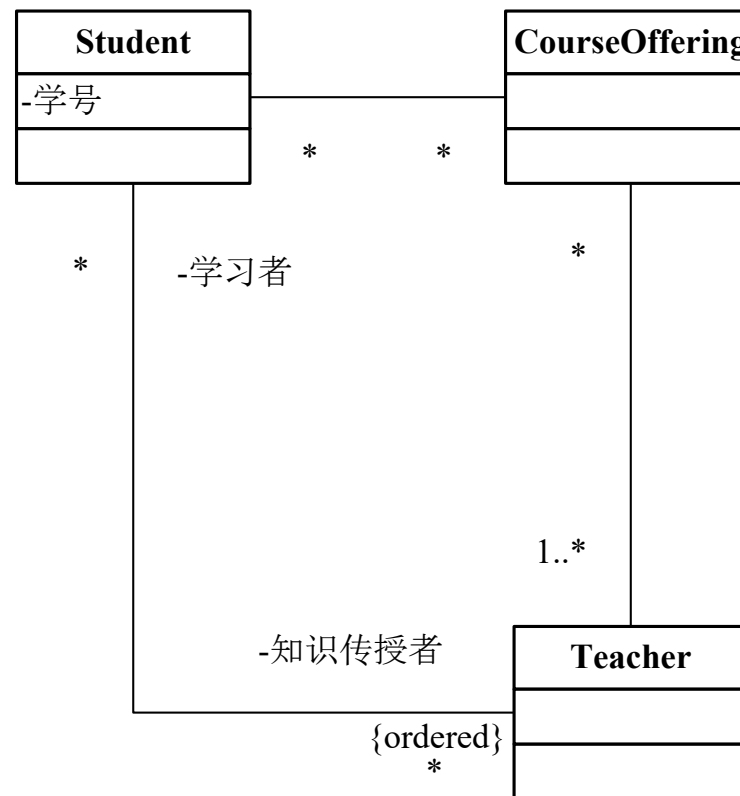


不同的关系采用不同的线条和箭头表示

类间关系-关联

□表示类间的逻辑联系

- ✓ **多重性**：位于关联端的类可以有多少个实例对象与另一端的类的单个实例对象相联系
- ✓ **角色名**：参与关联的类对象在关联关系中扮演的角色或发挥的作用
- ✓ **约束特性**：针对参与关联的对象或对象集的逻辑约束



类间关系-聚合与组合

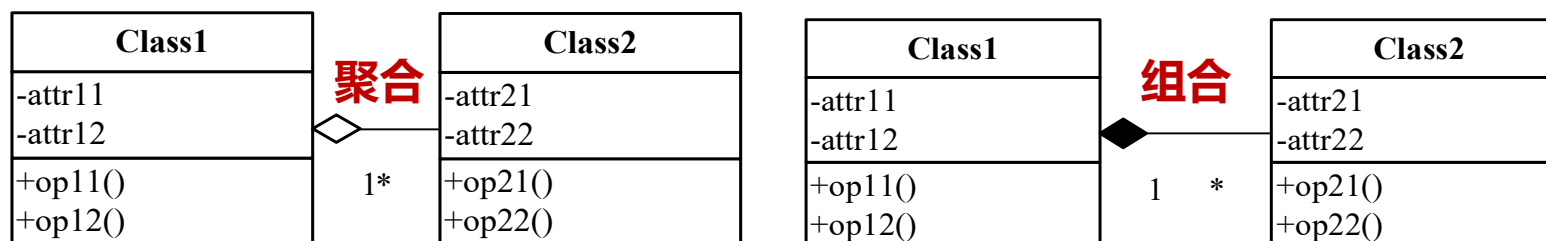
□聚合关系(Aggregation)

- ✓ **部分类**对象是多个**整体类**对象的组成
- ✓ 示例：一名学生可同时参与多个兴趣小组

□组合关系(Composition)

- ✓ **部分类对象只能位于一个整体类对象中**
- ✓ 一旦整体类对象消亡，部分类对象也不能苟活

举例说明组合和聚合的差异性



注意箭头图符

特别注意图符的画法和要求

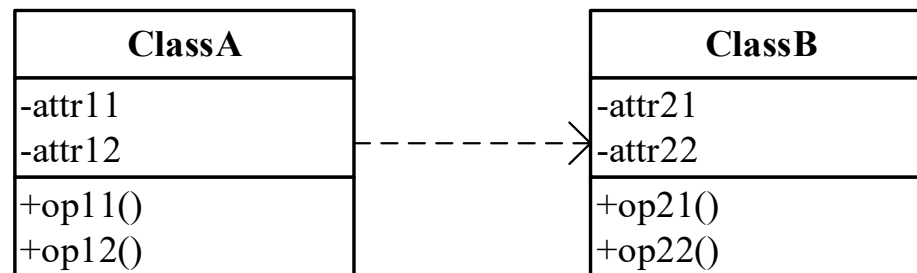
类间关系-依赖

□依赖关系

✓有**语义上关系**且一个类对象变化会导致另一类对象作相应修改

□依赖形式

- ✓**使用**: A类使用B类的某项服务
- ✓**追踪**: B类的变化导致A类的变化
- ✓**精化**: A类是在B类基础上引进设计、决策等形成
- ✓**实现**: A类给出了B类元素的实现
- ✓**派生**: A类可通过B类推导或计算出



注意箭头图符

特别注意图符的画法和要求

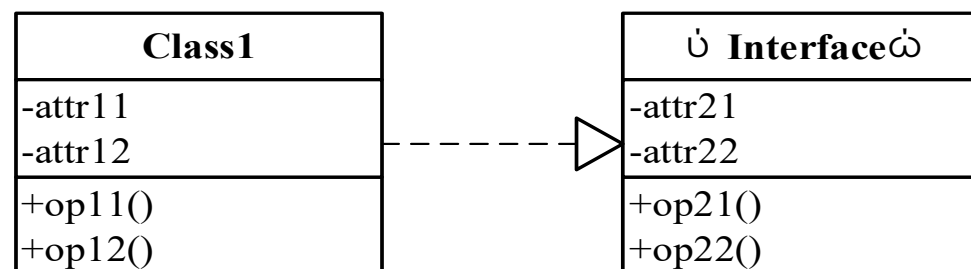
类间关系-实现

□表示一个类实现了另一个类中定义的对外接口

✓特殊依赖关系

□典型应用

✓一个具体类实现一个接口

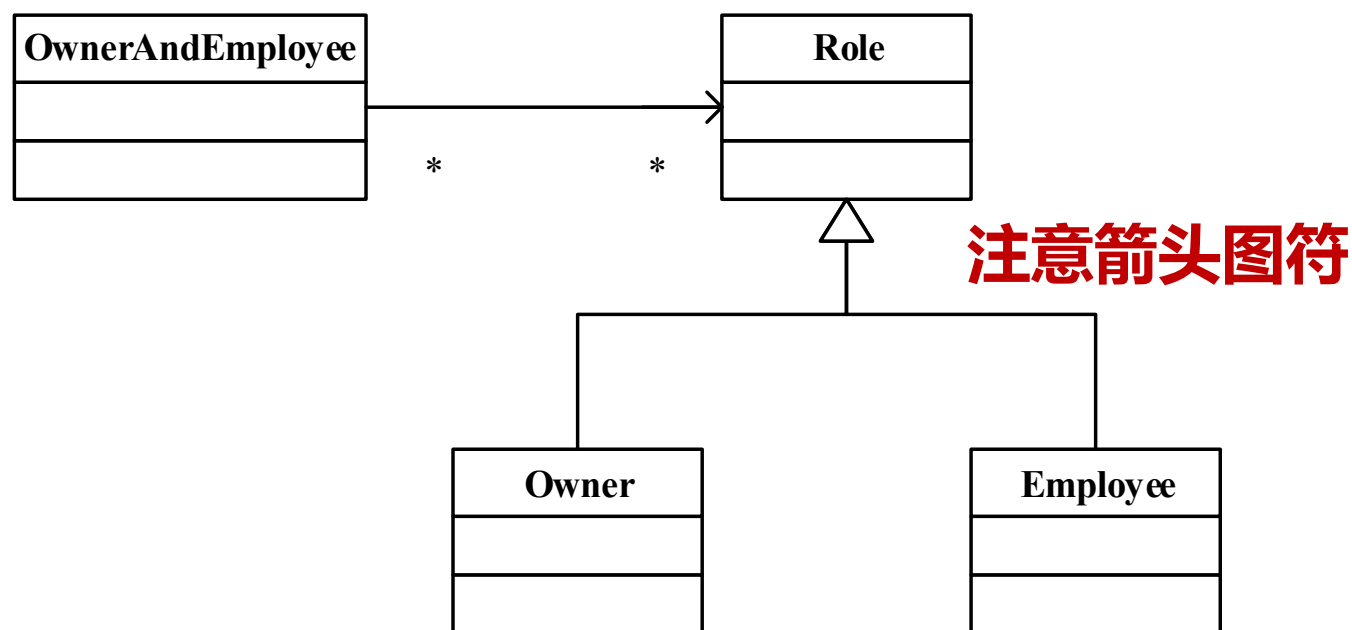


注意箭头图符

特别注意图符的画法和要求

类间关系-继承

- 子类**继承**父类所有可继承的特性，且可通过**添加**新特性或覆盖（**override**）父类中的原有特性
- 父类抽象多个子类中公共特性，从而避免冗余、简化操作



特别注意图符的画法和要求

注意事项

□用**单数名词**来描述类名，少用缩写

□按照**方向**表示类间关系

- ✓垂直方向表示继承关系

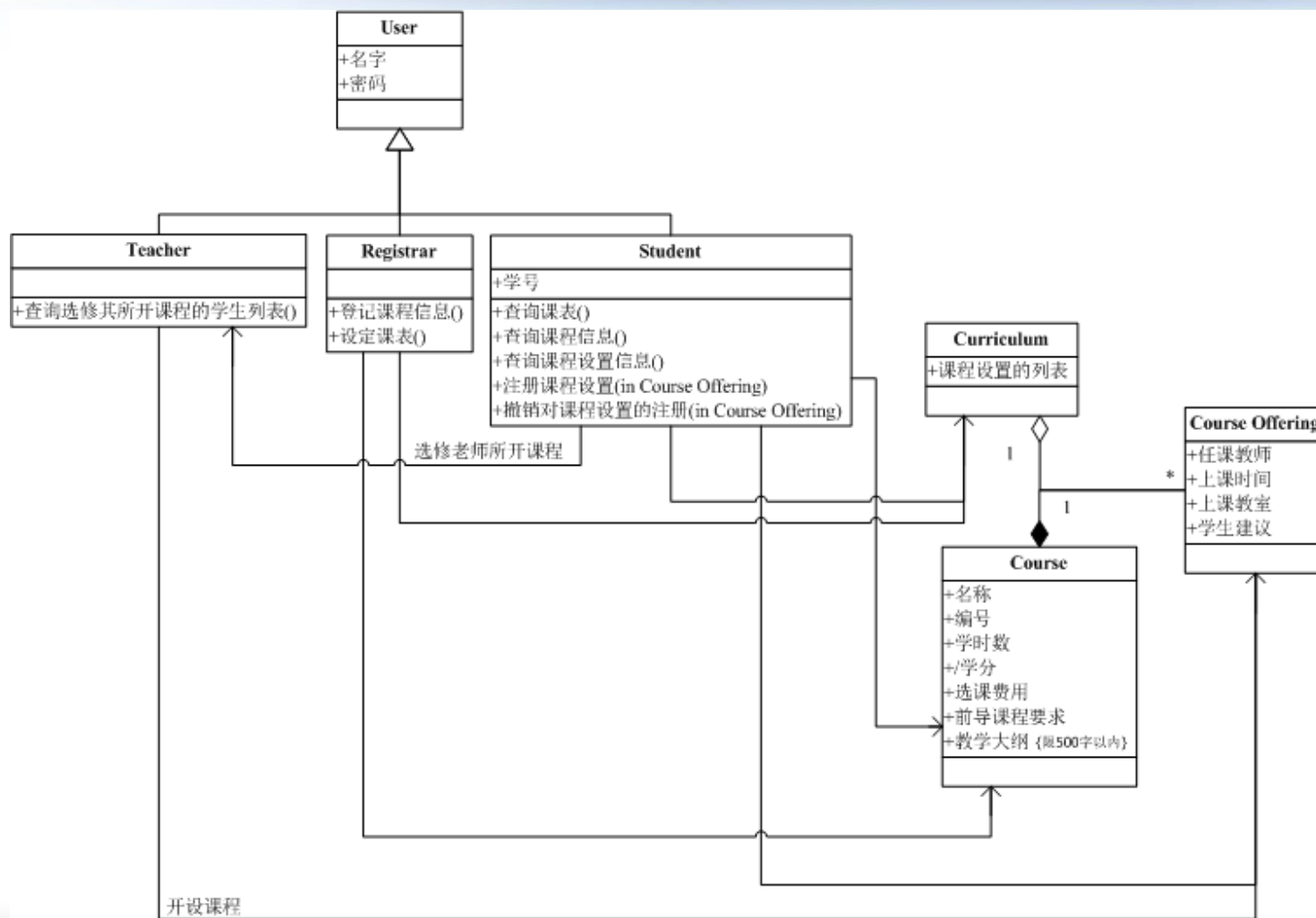
- ✓水平方向表示关联、聚合、组合、依赖、实现关系

□注意画图**位置**

- ✓关联名应位于关联边的居中位置

- ✓多重性、角色名、约束特性等应靠近关联端

示例：类图



注意图符及其
位置和方向

3.1.3 对象图

□功效

- ✓系统中的对象在运行过程中的瞬时快照
- ✓结点表示对象，边表示对象间的链接

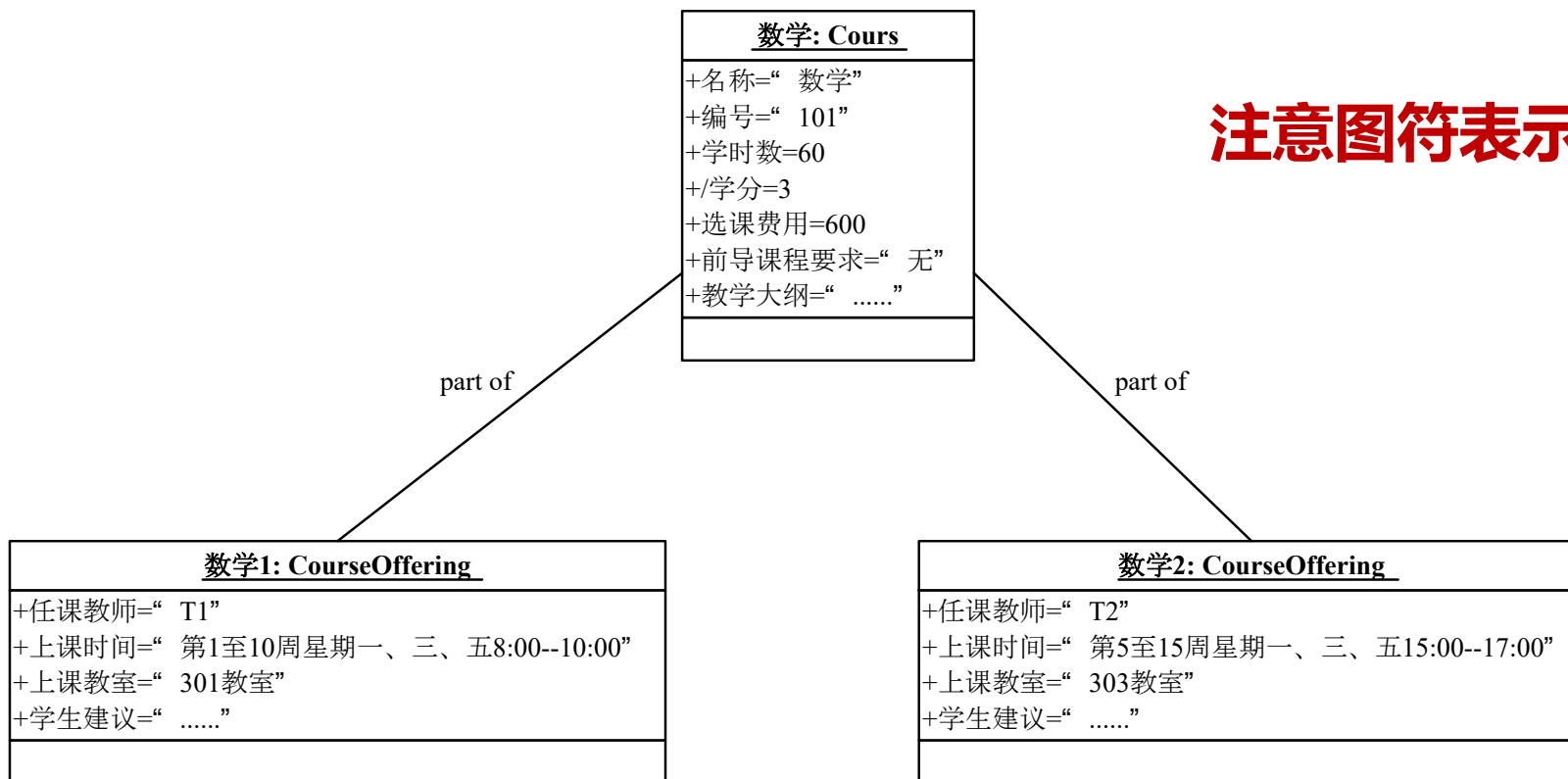
□类图在系统的运行过程中某个时刻点上或某一时间段内的 实例化样本

- ✓类图中的一个类在对象图中可表现为多个活跃的对象实例
- ✓对象图的链接边是类图中**关联**边的实例化
- ✓类图中的其他边，如**继承**、**依赖**等在对象图中则无从表现

□对象图是一种**静态瞬时快照**，归于静态视图范畴

对象图示例

注意图符表示



注意对象图和类图的区别

何时用对象图

□某些特定的观察点，需要分析系统中的对象情况

- ✓系统创建之时
- ✓某些对象消亡之前
- ✓.....

□通常而言，对象图不常用

**对象图 与 类图：
区别与联系**

状态图

视点	图 (diagram)	说明
结构	包图 (package diagram)	从包层面描述系统的静态结构
	类图 (class diagram)	从类层面描述系统的静态结构
	对象图 (object diagram)	从对象层面描述系统的静态结构
	构件图(component diagram)	描述系统中构件及其依赖关系
行为	状态图(statechart diagram)	描述状态的变迁
	活动图(activity diagram)	描述系统活动的实施
	通信图(communication diagram)	描述对象间的消息传递与协作
	顺序图(sequence diagram)	描述对象间的消息传递与协作
部署	部署图 (deployment diagram)	描述系统中工件在物理运行环境中的部署情况
用例	用例图 (use case diagram)	从外部用户角度描述系统功能

3.1.4 状态图

□功效

- ✓描述实体（对象、系统）在**事件**刺激下的**反应式动态行为**及其导致的**状态变化**
- ✓刻画了实体的可能状态、每个状态下可响应事件、响应动作、状态迁移

□图的构成

- ✓**节点**：状态
- ✓**边**：迁移，即状态间因事件刺激而触发的状态变化

状态图的基本概念

□ **状态**: **对象属性取值**构成的一个约束条件

✓ 相同状态下对象对事件的响应行为完全一样

□ **事件**: **某时刻点发生、需要关注的瞬时刺激或触动**

✓ **消息型事件(同步)**: 其他对象发来消息

✓ **信号型事件(异步)**: 其他对象传来异步信号

✓ **时间型事件**: 到达特定时间点

✓ **条件型事件**: 对象属性取值满足特定条件

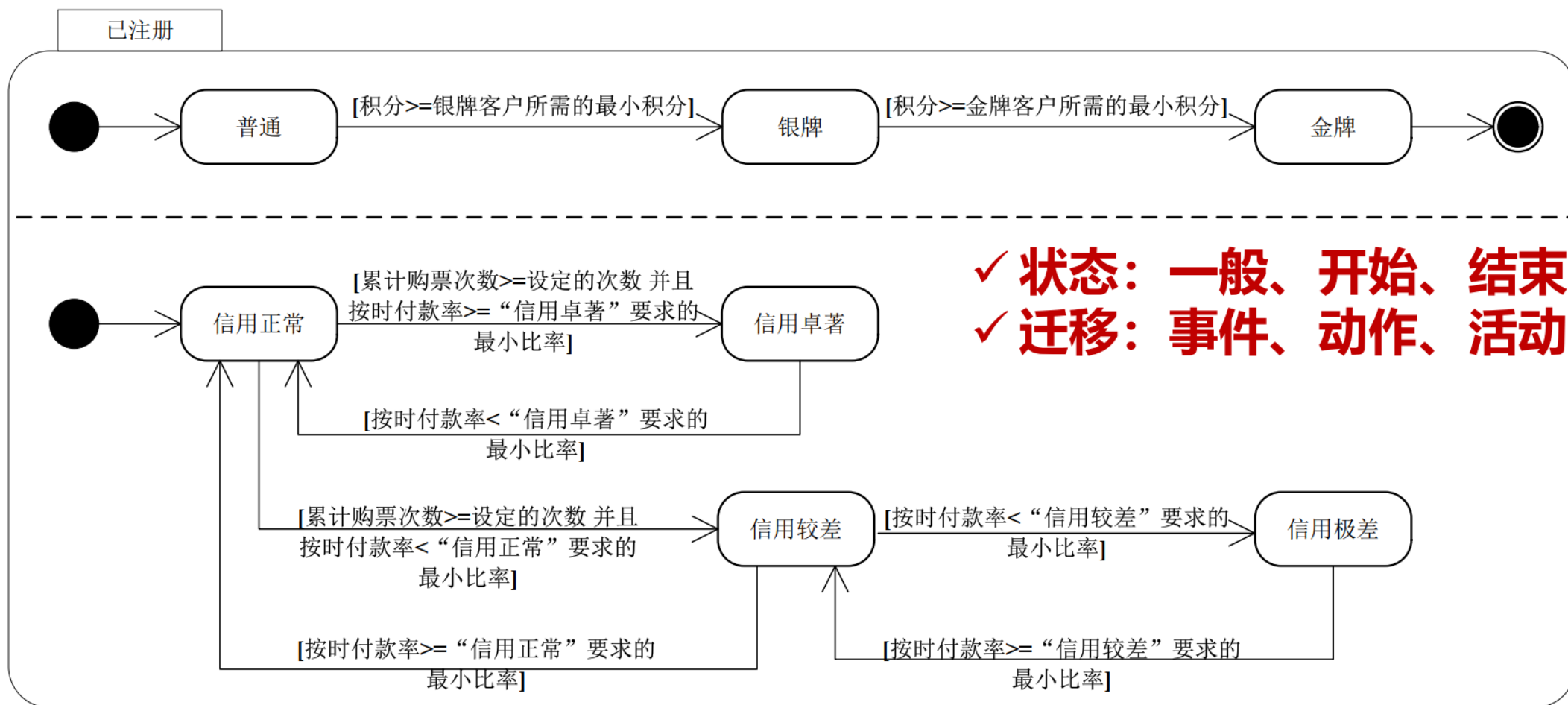
□ **动作(action)**

✓ 计算过程, **位于迁移边上**, 简单, 执行时间短

□ **活动(activity)**

✓ 计算过程, **位于状态中**, 复杂、执行时间长

示例：状态图



复合状态

- 状态 s 是由子状态 $s_1, \dots, s_n$ 经**AND (与) 逻辑复合**而成
是指, 对象处于状态 s 当且仅当对象同时处于 $s_1, \dots, s_n$
- 状态 s 由子状态 $s_1, \dots, s_n$ 经**XOR (异或) 逻辑复合**而成
是指, 对象处于状态 s 当且仅当对象处于 $s_1, \dots, s_n$ 中的某个子状态

状态的表示

□由状态名及可选的入口活动、出口活动、do活动、内部迁移构成

- ✓一旦对象经迁移边从其他状态进入本状态，那么本状态**入口活动**将被执行
- ✓一旦对象经迁移边从本状态进入其他状态，那么本状态的**出口活动**将被执行
- ✓**do活动**是当对象进入本状态并执行完入口活动（如果有的话）后应该执行的活动。
- ✓**内部迁移**不会引起对象状态的变化，所以它虽有源状态（包含此迁移的状态），但没有目标状态

迁移的表示

□迁移标注为“[事件] [监护条件] [/ 动作]”

- ✓“**事件**”表示触发此次状态变迁的事件
- ✓“**监护条件**”表示约束状态迁移真正发生的条件表达式
- ✓“**动作**”表示状态迁移期间应当执行的动作 (action)
- ✓在监护条件和动作中均可引用触发事件参数和对象属性

□迁移表示为状态节点间的有向边

- ✓自迁移是指源状态与目标状态相同的特殊外部迁移

状态图绘制原则

□绘图

- ✓状态节点按行上下对齐，按列左右对齐
- ✓迁移边绘制成水平线段、垂直线段或者由水平、垂直两种线段组合而成的折线
- ✓避免斜线

□初态应位于左上角，终态尽量靠近右下角

□迁移边上的文本标注应靠近源状态

3.2 分析和确定软件需求优先级

- 分析软件需求**重要性**
- 分析软件需求**优先级**
- 确定用例分析和实现的**次序**

3.2.1 分析软件需求重要性

□从用户和客户的视角，软件需求的**重要性**是不一样的

□核心软件需求

✓在解决问题方面起到举足轻重的作用，提供了软件系统所特有的功能和服务，体现了软件系统的特色和优势

□外围软件需求

✓提供了次要、辅助性的功能和服务

示例：“空巢老人看护软件”的需求重要性

序号	用例名称	用例标识	重要性
1	监视老人	UC-MonitorElder	核心
2	自主跟随老人	UC-FollowElder	核心
3	获取老人信息	UC-GetElderInfo	核心
4	检测异常状况	UC-CheckEmergency	核心
5	通知异常状况	UC-NotifyEmergency	核心
6	控制机器人	UC-ControlRobot	外围
5	视频/语音交互	UC- A&VInteraction	外围
6	提醒服务	UC-AlertService	外围
9	用户登录	UC-UserLogin	外围
10	系统设置	UC-SetSystem	外围

示例：“Mini-12306”的需求重要性

- “注册”、“登录”、“查询车次”、“购票”、“支付”、“退票”和“改签”这7项功能重要性高
- “系统设置”功能的重要性低

3.2.2 分析软件需求优先级

□用户对实现这些软件需求的**优先级**要求也不一样

- ✓有些软件需求需要优先实现，尽早交付给用户使用，以发挥其价值；有些软件需求可以滞后实现，晚点交付使用

□优先级的考虑因素

- ✓结合软件项目开发的具体约束，考虑不同软件需求的重要性，确定软件需求的实现优先级，确保在整个迭代开发中有计划、有重点地实现软件需求
- ✓按照软件需求的**重要性**来确定其**优先级**
- ✓按照用户的**实际需要**来确定软件需求的**优先级**

示例：确定“空巢老人看护软件”的需求优先级

用例名称	用例标识	重要性	优先级
监视老人	UC-MonitorElder	核心	高
获取老人信息	UC-GetElderInfo	核心	高
检测异常状况	UC-CheckEmergency	核心	高
通知异常状况	UC-NotifyEmergency	核心	高
自主跟随老人	UC-FollowElder	核心	高
视频/语音交互	UC- A&VInteraction	外围	中
控制机器人	UC-ControlRobot	外围	低
提醒服务	UC-AlertService	外围	低
用户登录	UC-UserLogin	外围	低
系统设置	UC-SetSystem	外围	低

示例：“Mini-12306”的需求优先级

- “注册”、“登录”、“查询车次”、“购票”、“支付”五项功能具有最高优先级
- “退票”和“改签”这项功能也很重要，优先级次之
- “系统设置”软件需求的优先级最低

3.2.3 确定用例分析和实现的次序

- **确保有序地开展需求分析、软件设计和实现工作，使得每次迭代开发有其明确的软件需求集，每次迭代开发结束之后可向用户交付他们所急需的软件功能和服务**
 - ✓ 结合软件开发的迭代次数、每次迭代的持续时间、可以投入的人力资源等具体情况
 - ✓ 充分考虑相关软件需求项的开发工作量和技術难度等因素，确定需求用例分析和实现的先后次序

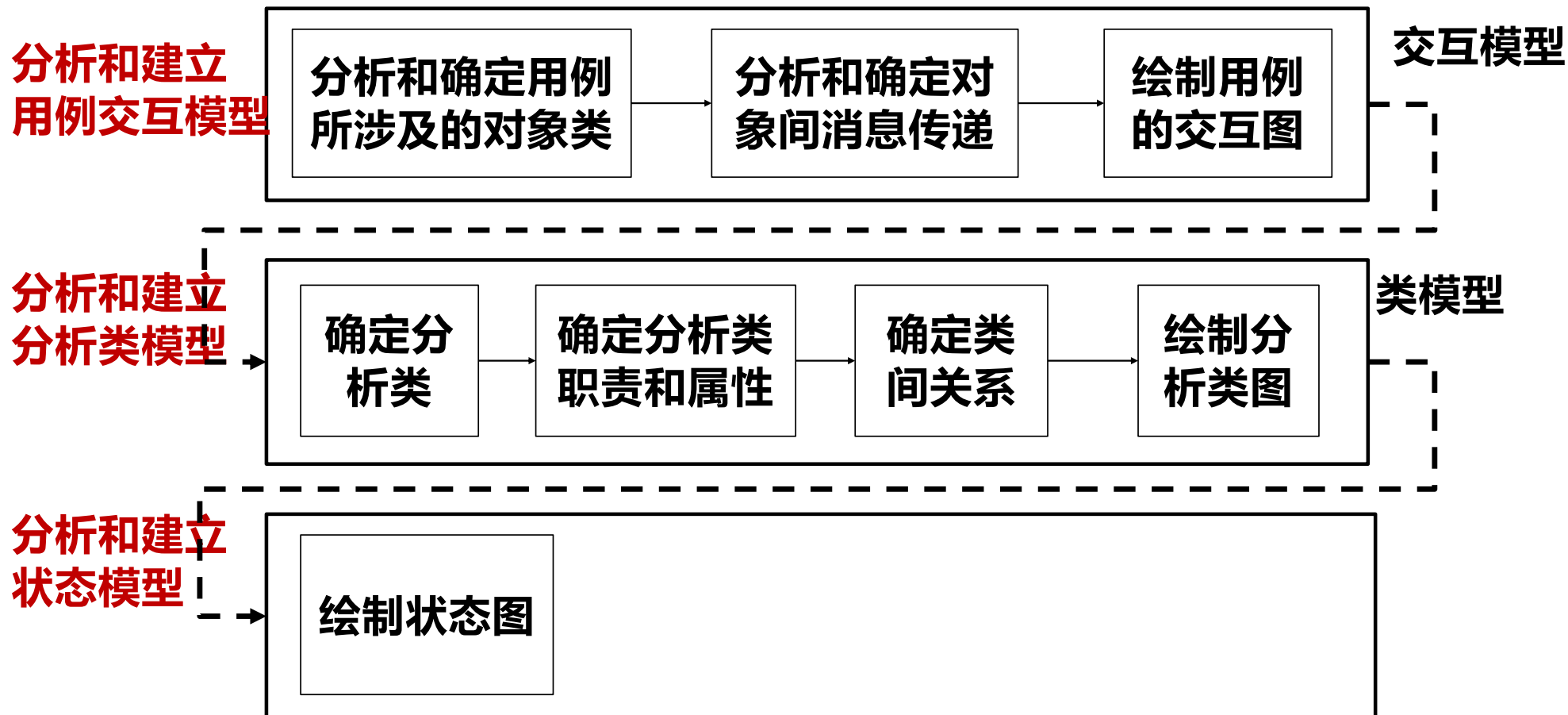
示例：“空巢老人看护软件”需求开发安排

用例名称	用例标识	优先级	迭代次序
监视老人	UC-MonitorElder	高	第一次迭代
获取老人信息	UC-GetElderInfo	高	
检测异常状况	UC-CheckEmergency	高	
通知异常状况	UC-NotifyEmergency	高	第二次迭代
自主跟随老人	UC-FollowElder	高	
视频/语音交互	UC- A&VInteraction	中	第三次迭代
控制机器人	UC-ControlRobot	低	
提醒服务	UC-AlertService	低	第四次迭代
用户登录	UC-UserLogin	低	
系统设置	UC-SetSystem	低	

示例：“Mini-12306”需求开发安排

序号	用例名称	用例标识	重要性	优先级	开发迭代次序
1	Register	UC-Register	核心	高	第一次迭代
2	Login	UC -Login	核心	高	
3	QueryTrain	UC -QueryTrain	核心	高	
4	BuyTicket	UC -BuyTicket	核心	高	
5	Pay	UC -Pay	核心	高	
6	RefundTicket	UC -RefundTicket	外围	中	第二次迭代
7	ChangeTicket	UC -ChangeTicket	外围	中	
8	SetSystem	UC -SetSystem	外围	低	第三次迭代

3.3 分析和建立软件需求模型的具体步骤



3.3.1 分析和建立用例的交互模型

□任务

✓分析和描述用例是如何通过一组**对象**之间的**交互**来完成的

□步骤

- ① 分析和确定用例所涉及的**对象及其类**
- ② 分析和确定对象之间的**消息传递**
- ③ 绘制用例的**交互图**

分析和确定用例所涉及的对象及其类

□ 软件需求用例的处理通常涉及三种不同类对象

✓ 边界类

✓ 控制类

✓ 实体类

□ 这些类是在用例分析阶段所识别并产生的，通常将它们称为分析类

边界类

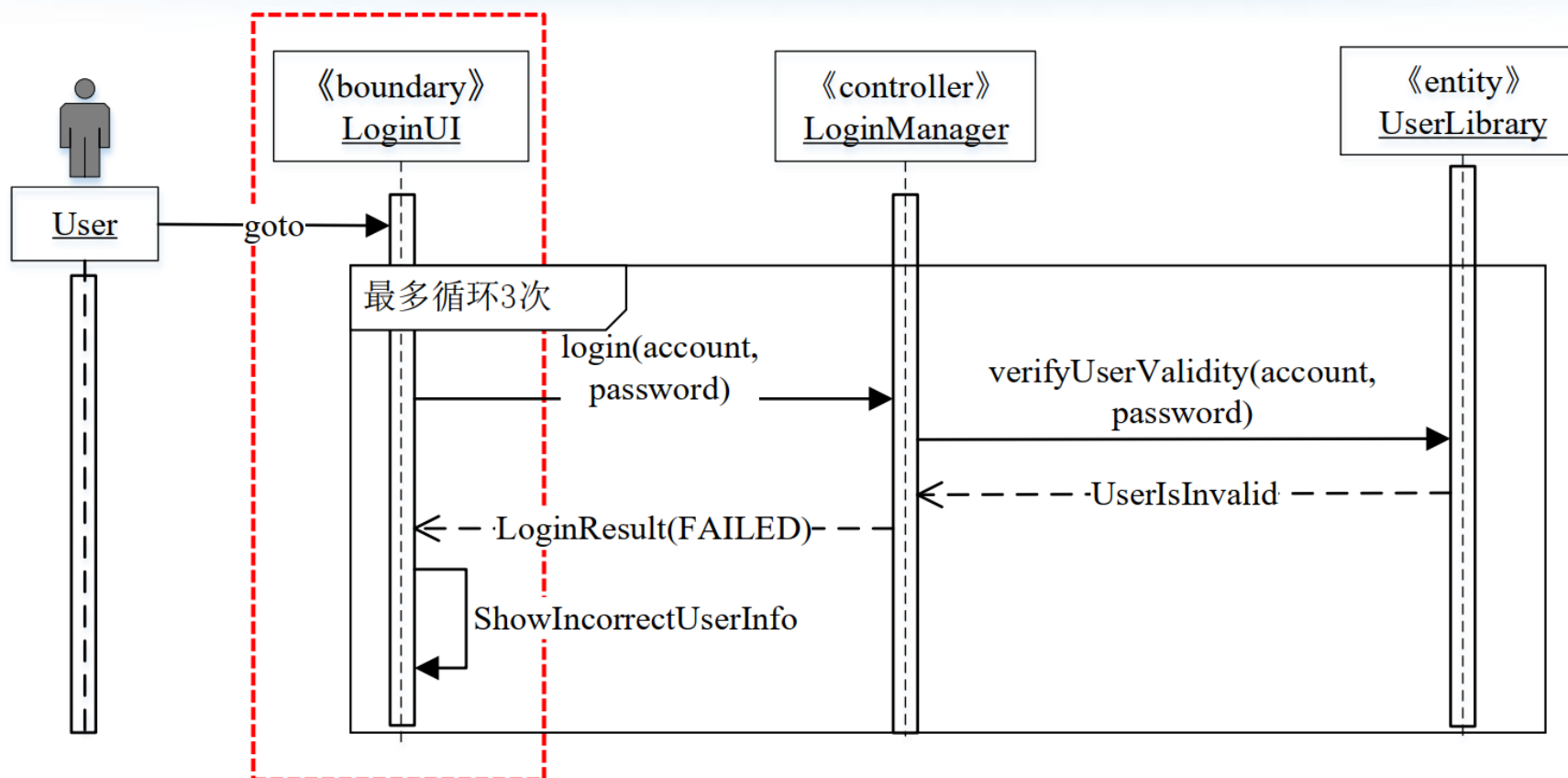
□何为边界类

- ✓每个用例或者由外部执行者触发，或者需要与外部执行者进行某种信息交互，因而用例的业务逻辑处理需要有一个类对象来负责
目标软件系统与外部执行者之间的交互
- ✓由于这些类对象**处于系统的边界**，需与系统外的执行者进行交互，因而将这些对象所对应的类称之为边界类

□边界类对象主要起到以下作用

- ✓**交互控制**，处理外部执行者的输入数据，或者向外部执行者输出数据
- ✓**外部接口**，如果外部执行者表现为其他的系统或者设备，那么边界类对象需要与系统之外的其他系统或设备进行信息交互

示例：边界类



边界类在顺序图中有何特点？



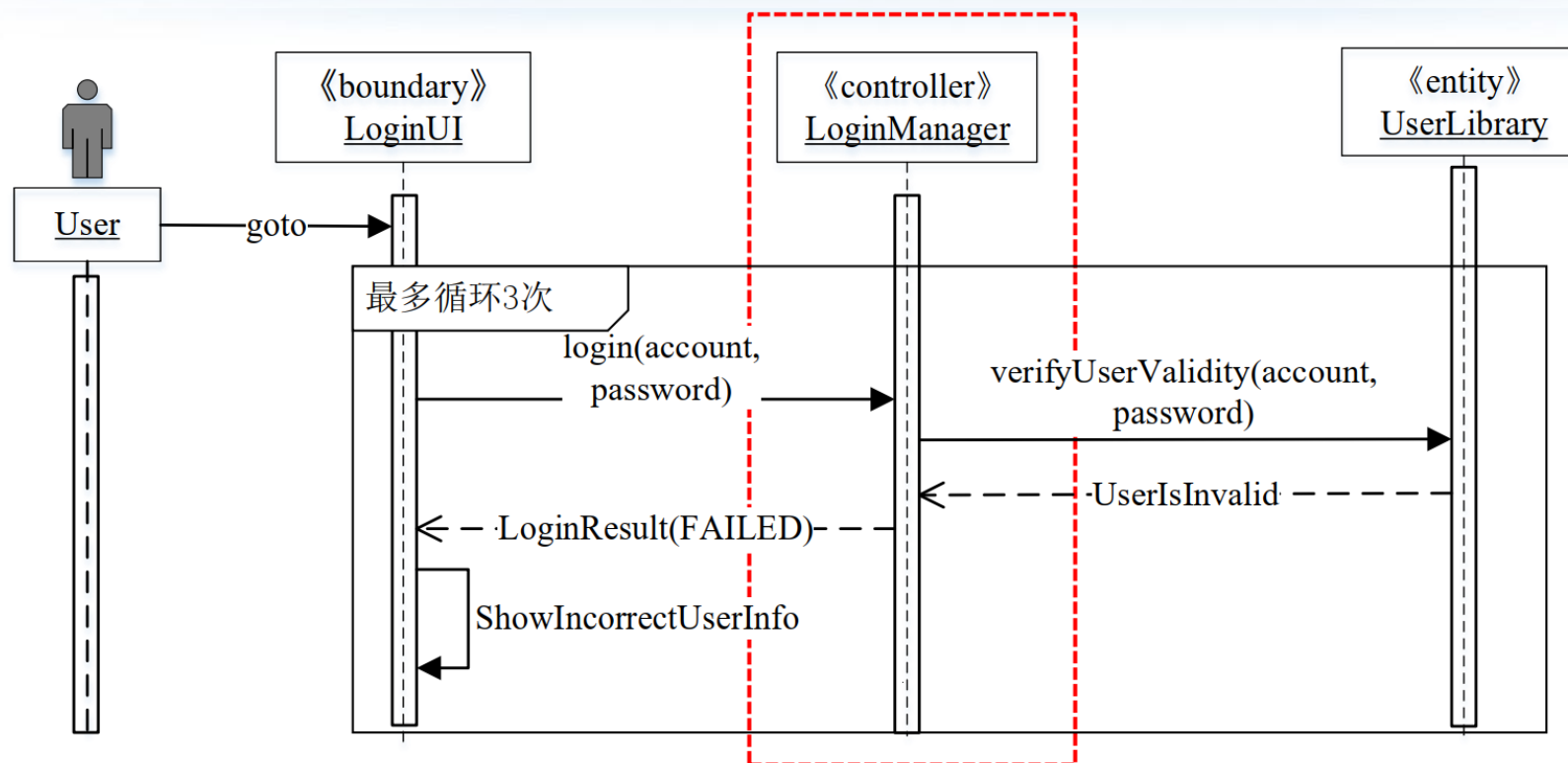
控制类

□控制类对象作为完成用例任务的主要协调者

- ✓负责处理边界类对象发来的任务请求，对任务进行适当的分解，并与系统中的其他对象进行协同，以控制他们共同完成用例规定的任务或行为

□一般而言，控制类并不负责处理具体的任务细节，而是负责分解任务，并通过消息传递将任务分派给其他对象类来完成，协调这些对象之间的信息交互

示例：控制类



控制类在顺序图中有何特点？

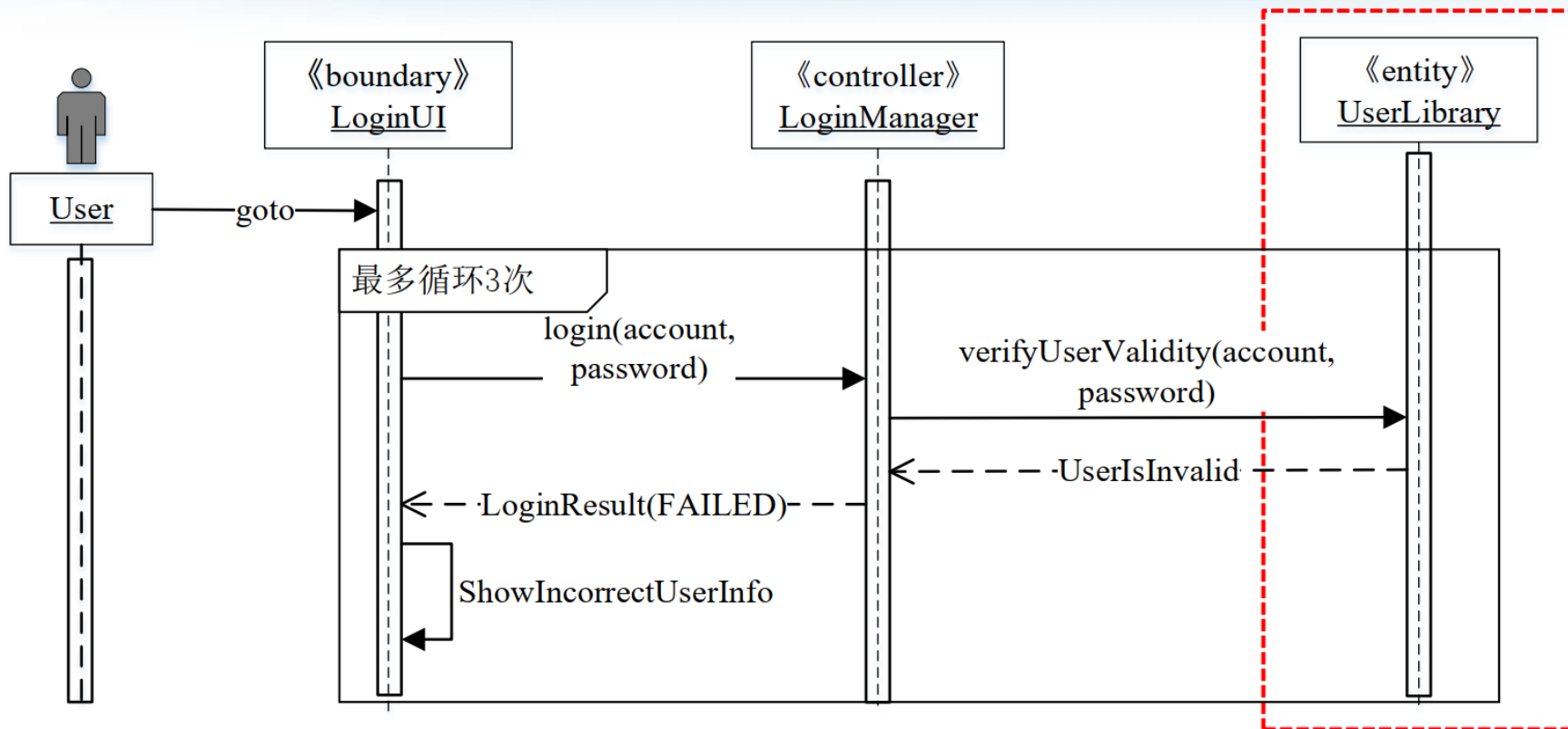


实体类

□用例所对应**业务流程中的所有具体功能**最终要交由具体的类对象来完成，这些类称之为**实体类**

- ✓一般地，实体类对象负责保存目标软件系统中具有持久意义的信息项，对这些信息进行相关的处理（如查询、修改、保存等），并向其他类提供信息访问的服务
- ✓实体类的职责是要**落实目标软件系统中的用例功能**，提供相应的业务服务

示例：实体类



实体类在顺序图中有何特点？



3.3.2 分析和确定对象之间的消息传递

□确定消息的名称

□确定消息传递的信息

确定消息的名称

□消息的名称

- ✓直接反映了对象间**交互的意图**，也体现了接收方对象所对应的类需**承担的职责和任务**，也即发送方对象希望接收方对象提供什么样的功能和服务
- ✓**意图**：请求、通知

□消息名称的表示

- ✓一般地，消息名称用**动名词**来表示
- ✓需求工程师应尽可能用**应用领域中通俗易懂的术语**来表达消息的名称和参数，以便于用户和需求工程师等能直观地理解对象间的交互语义

确定消息传递的信息

□对象间的交互除了要表达消息名称和交互意图之外，在许多场合还需要提供必要的**交互信息**

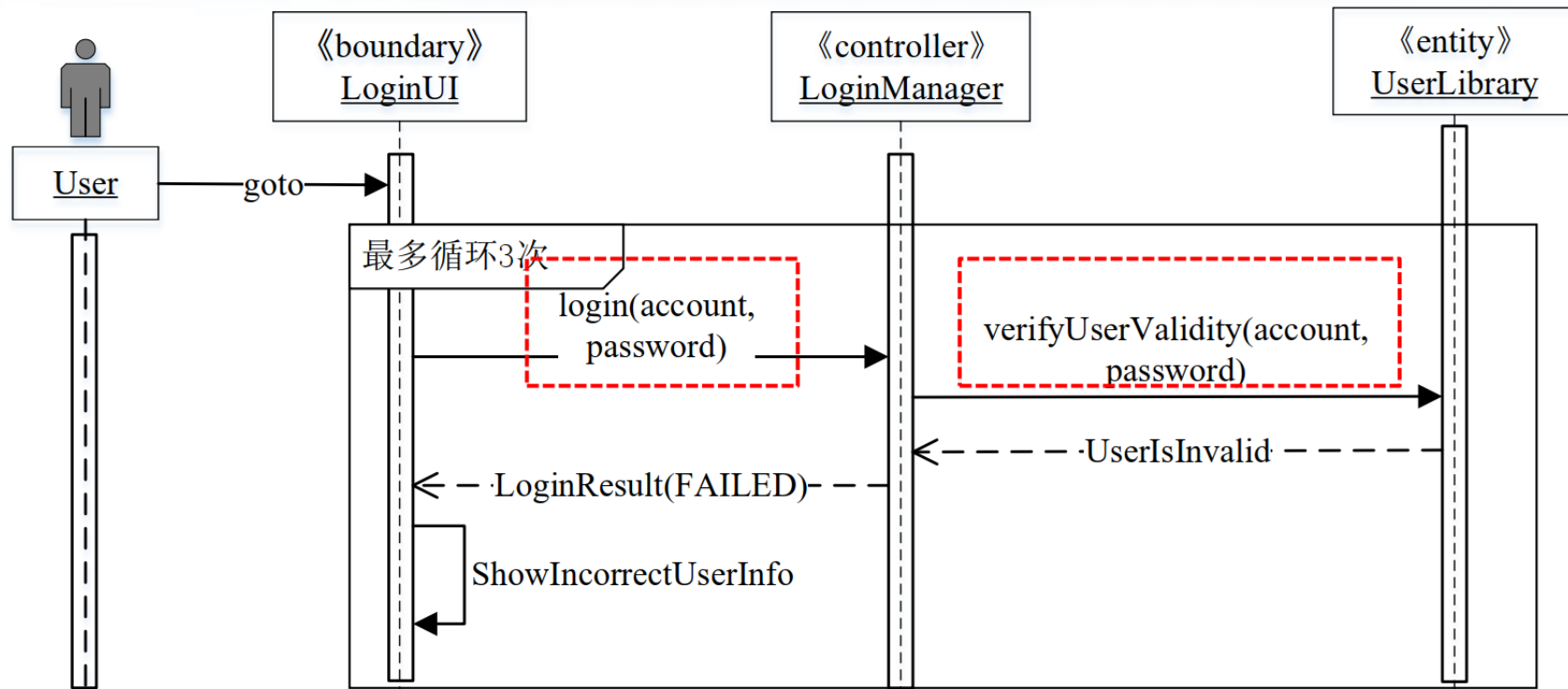
✓这些信息通常以消息参数的形式出现，也即一个对象在向另一个对象发送消息的过程中，需要提供必要的参数，以向目标对象提供相应的信息

✓**信息**：通知和请求的内容

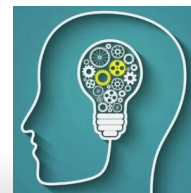
□在构建用例的交互图过程中，如果用例的业务流程能够明确相应的交互信息，那么就需要确定消息需附带的信息

□通常，消息参数用名词或名词短语来表示

示例：消息的名称和信息



消息的意图和信息是什么？



3.3.3 绘制用例的交互图

□绘制顺序图的策略

- ✓用例的外部执行者应位于图的最左侧，紧邻其右的是用户界面或外部接口的边界类对象，再往右是控制类对象，控制类的右侧应放置实体类对象，它们的右侧是作为外部接口的边界类对象
- ✓对象间的消息传递采用**自上而下的布局方式**，以反映消息交互的时序先后

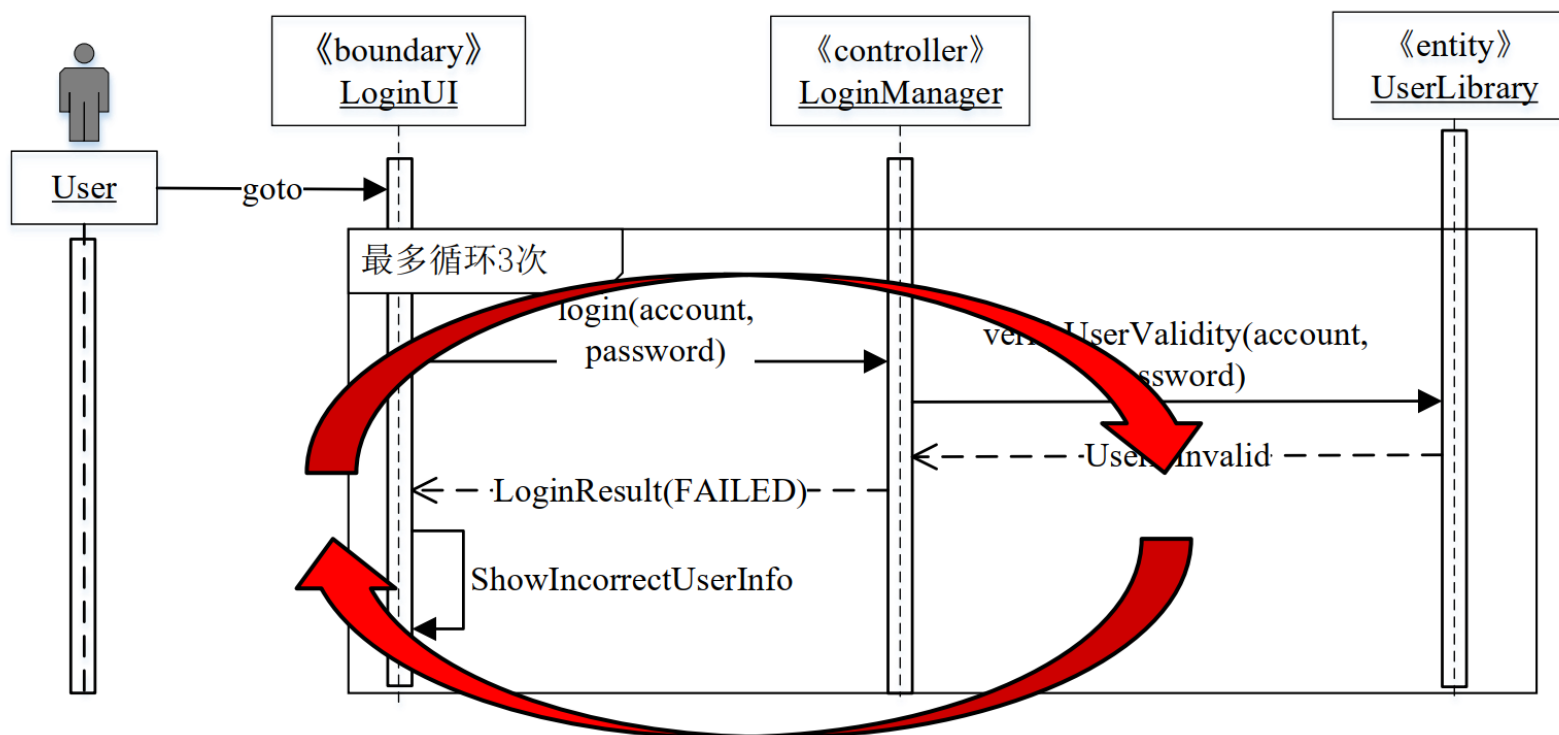
□一个用例至少构造一个交互图，以刻画用例的行为模型

- ✓对于较为简单的用例，为其构造一个交互图就足够了
- ✓对于较复杂的用例而言，需要为此用例绘制多张交互图，每张交互图刻画了用例在某种特定场景下的交互动作序列

用例交互图的工作流程

- ① 外部执行者与边界类对象进行**交互以启动用例**的执行
- ② 边界类对象接收外部执行者提供的信息，将信息从外部形式转换为内部形式，并通过**消息传递将相关信息发送给控制类对象**
- ③ 控制类对象根据业务逻辑处理流程，产生和分解任务，与相关的实体类对象进行**交互以请求完成相关的任务**，或者向实体类对象提供业务信息，或者请求实体类对象持久保存业务逻辑信息，或者请求获得相关的业务信息
- ④ 实体类对象实施相关的行为后，向控制类对象**反馈信息处理结果**
- ⑤ 控制类对象处理接收到的信息，将处理结果**通知边界类对象**
- ⑥ 边界类对象对接收到的处理结果信息进行必要的分析，将其从内部形式转换为外部形式，通过界面将处理结果**展示给外部执行者**

示例：顺序图的工作流程

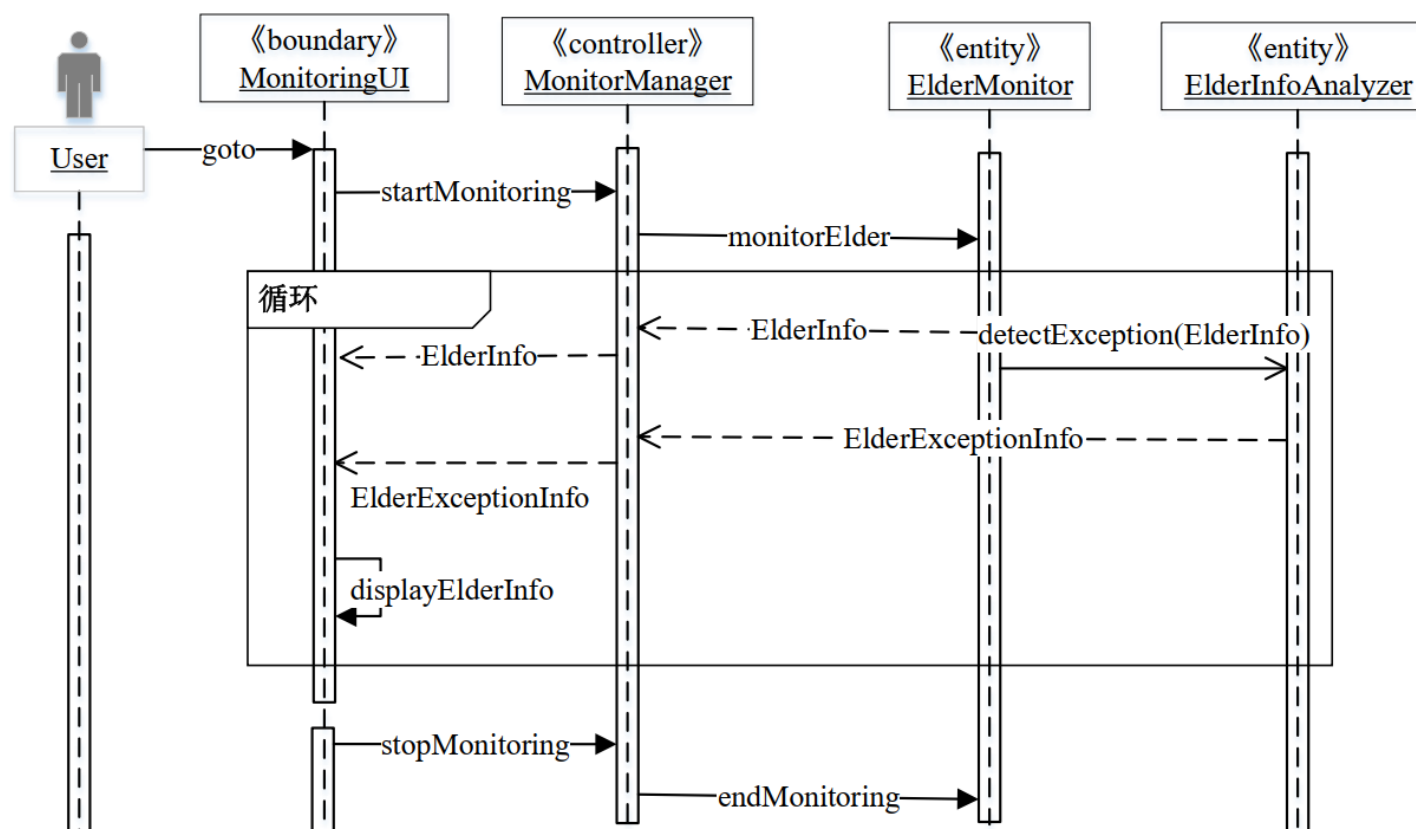


顺序图的工作流程有何特点？



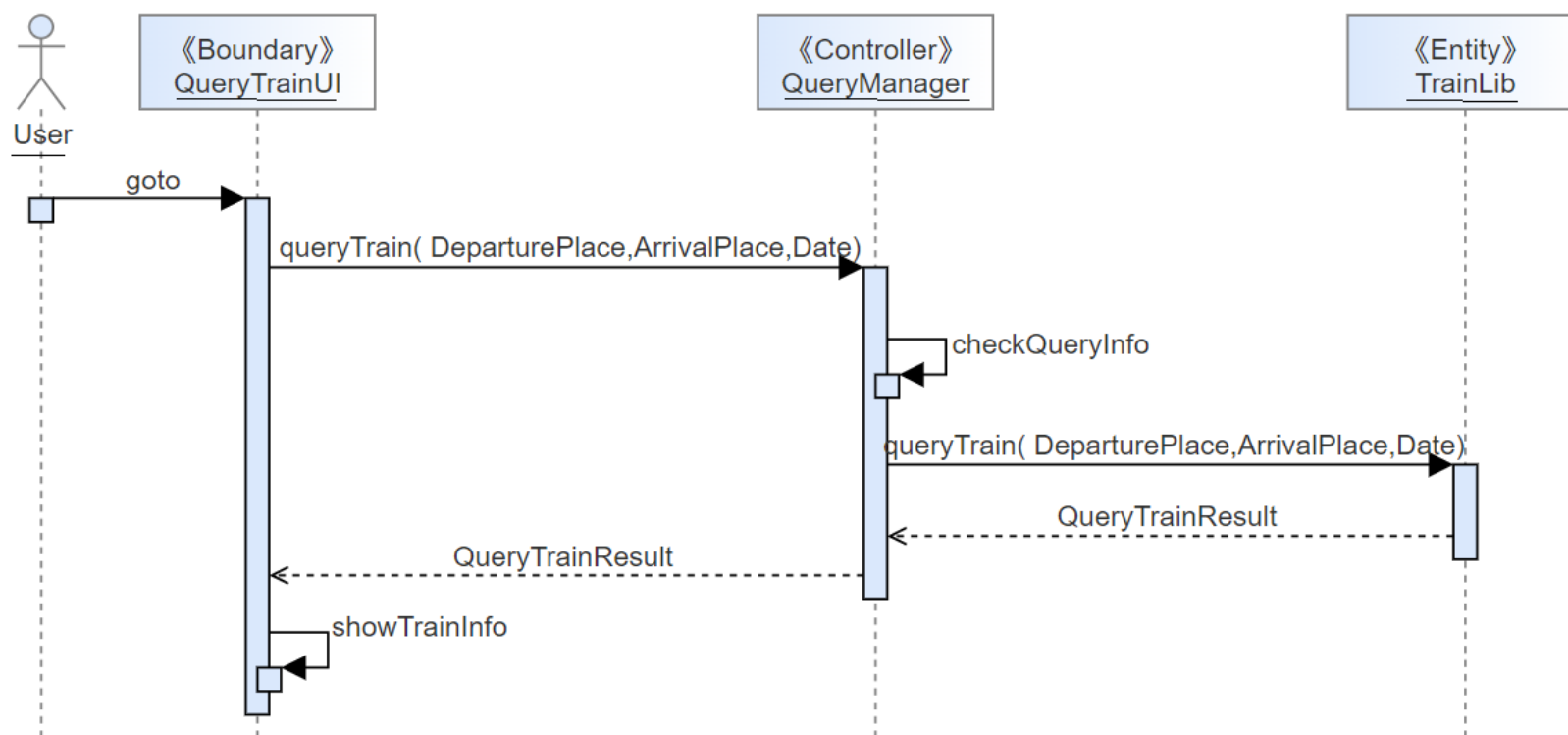
示例：“空巢老人看护”中用例的交互模型

□UC-MonitorElder “监视老人”用例的交互模型

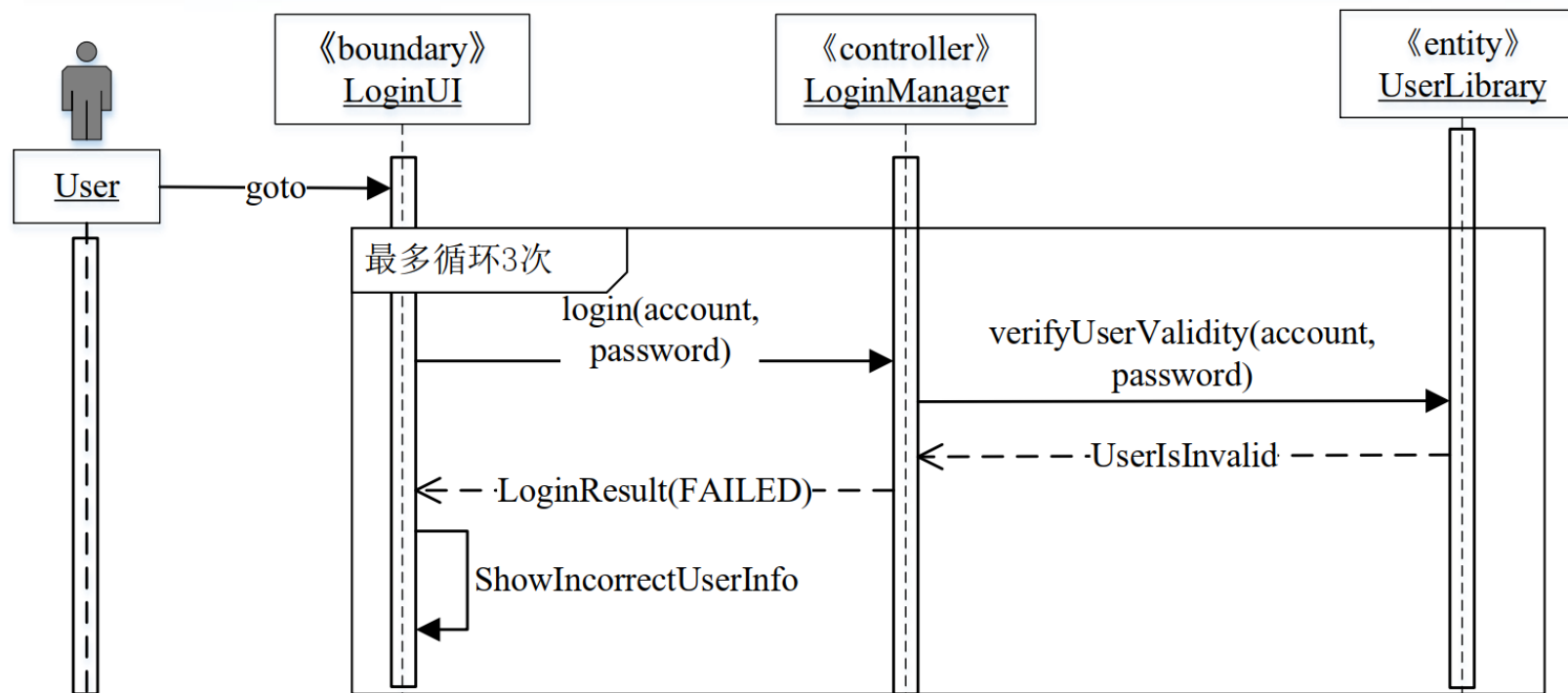


示例：“Mini-12306” 中用例的交互模型

□ “QueryTrain (查询车次)” 用例的顺序图



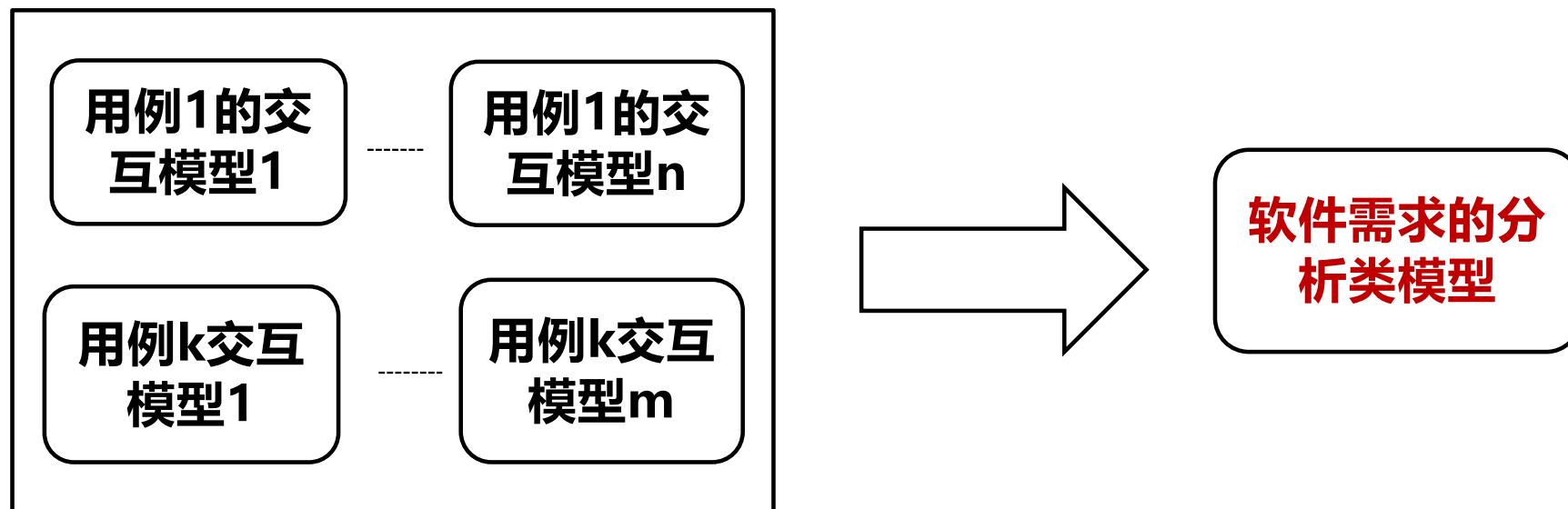
思考和讨论



一个用例用一张顺序图描述够吗？为什么？



3.4 分析和建立软件需求的分析类模型



分析和建立分析类模型的步骤

- ① 确定分析类
- ② 确定分析类的职责
- ③ 确定分析类的属性
- ④ 确定分析类之间的关系
- ⑤ 绘制分析类图

3.4.1 确定分析类

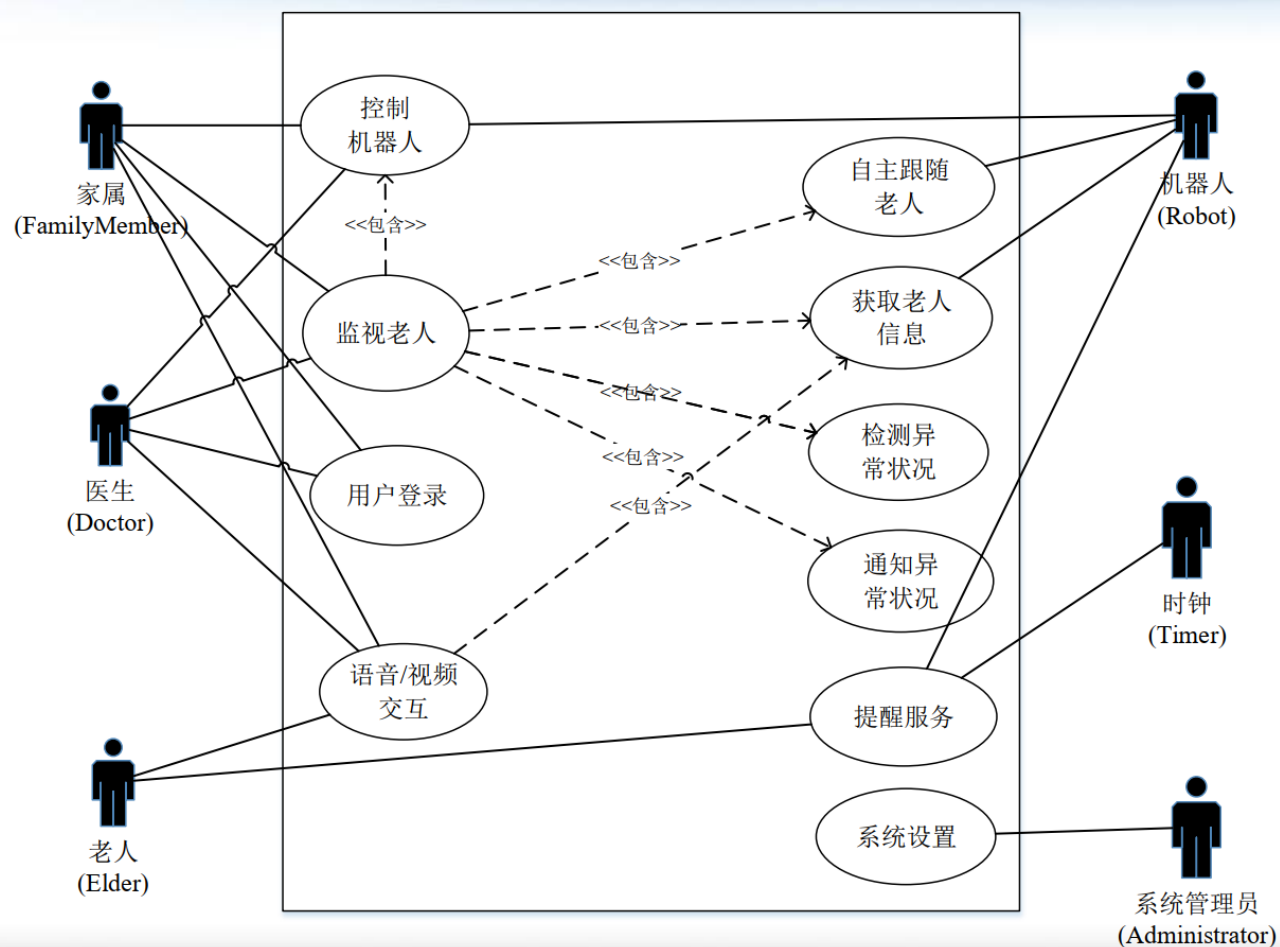
□有哪几种分析类？



□如何确定分析类？

- ✓用例模型中的**外部执行者**应该是分析类图中的类
- ✓在各个用例的顺序图，如果该图中出现了某个**对象**，那么该**对象所对应的类**属于分析类，应出现在分析类图中

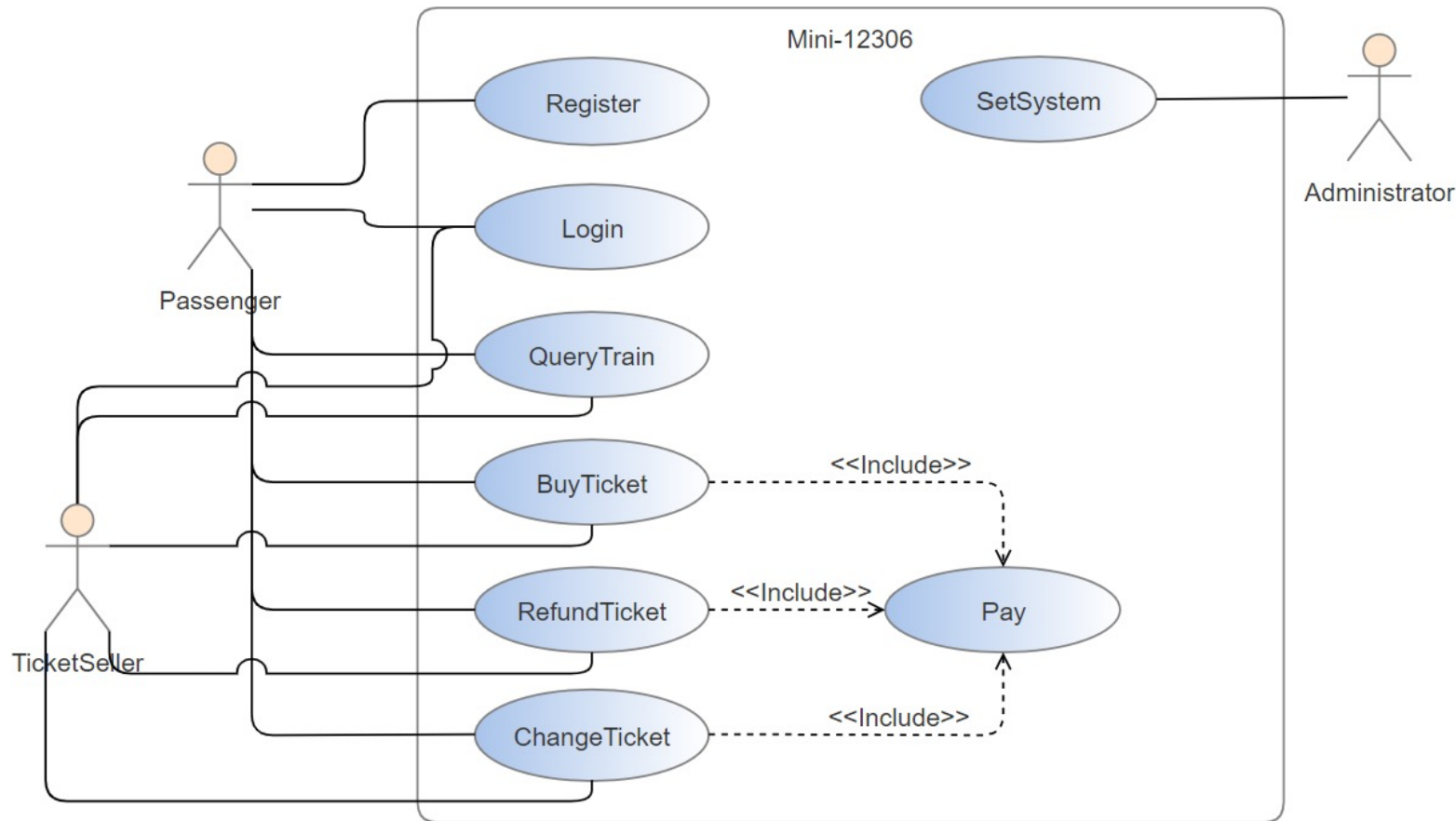
示例：根据用例图来确定分析类



有哪些分析类?



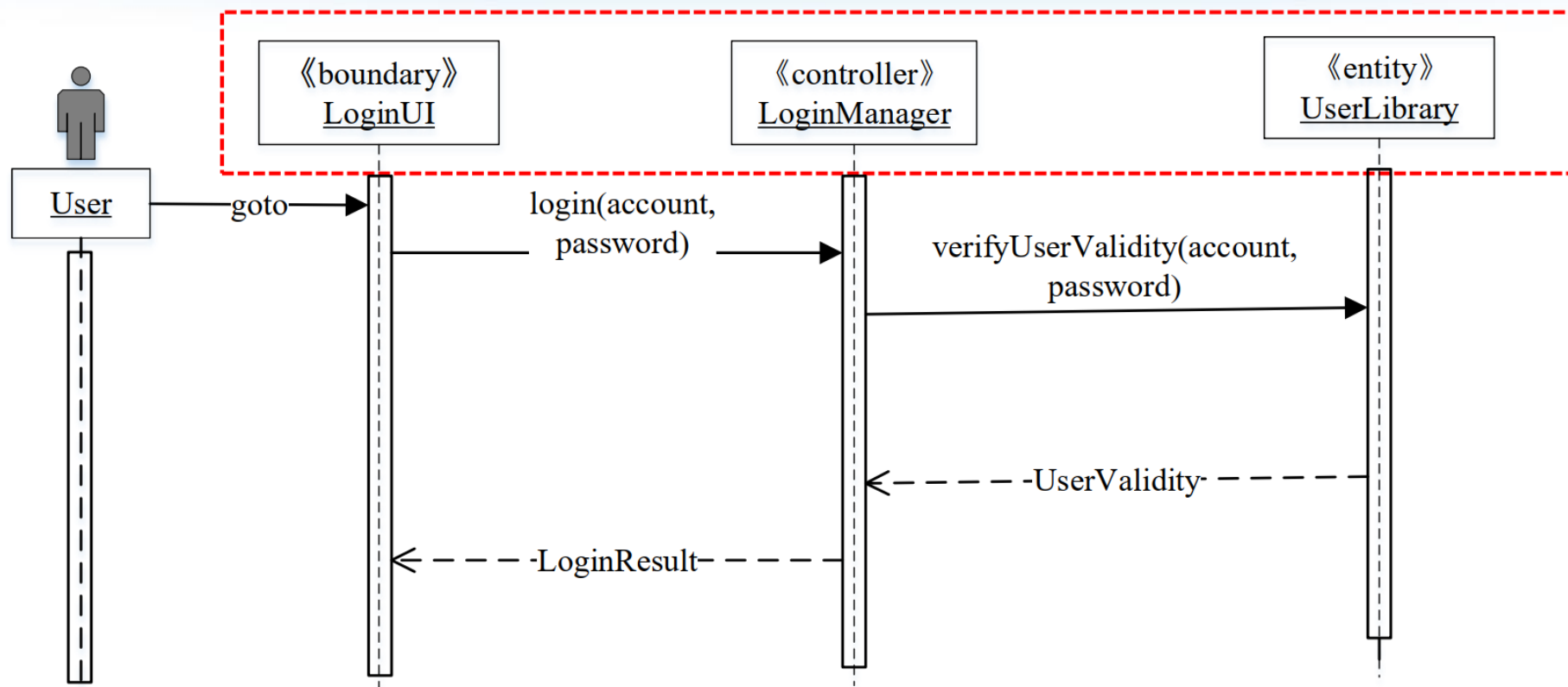
示例：根据用例图来确定分析类



有哪些分析类？



示例：根据交互图来确定分析类



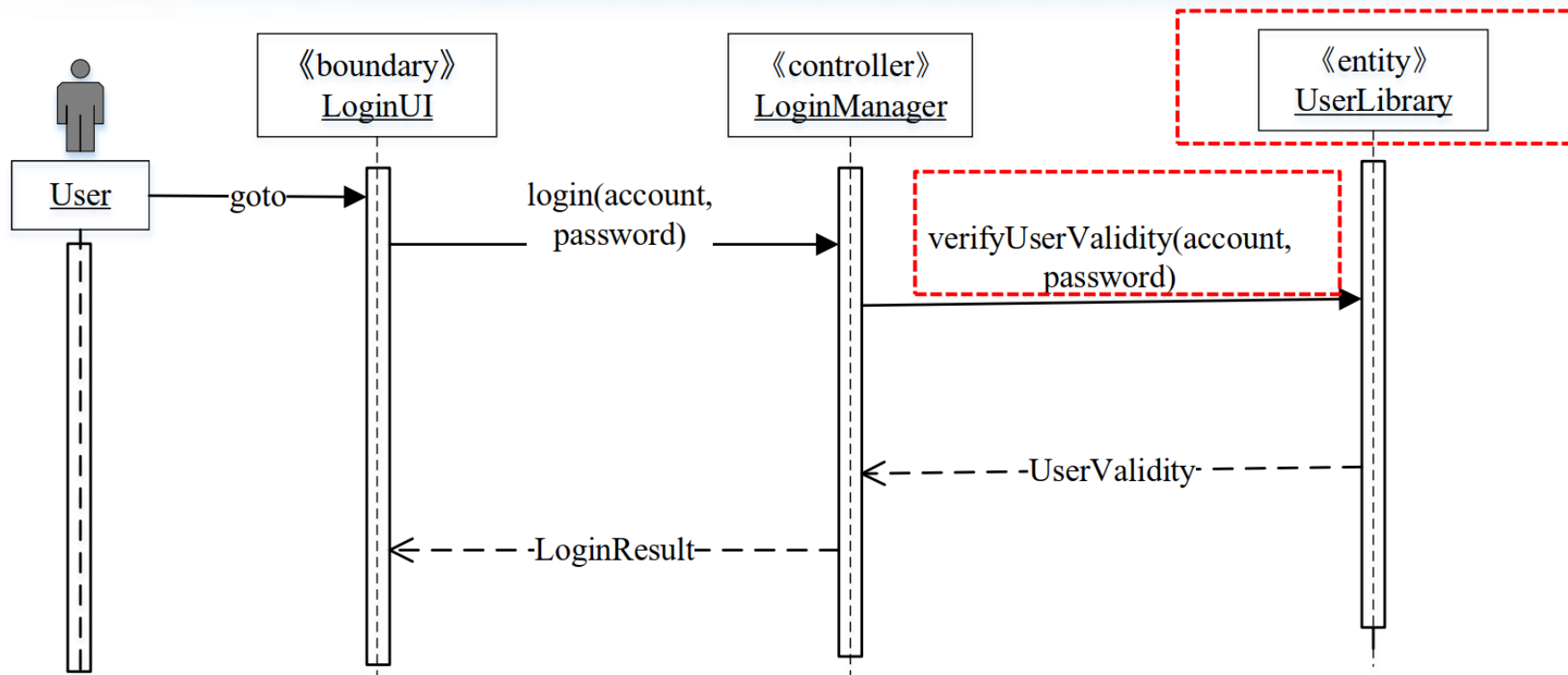
有哪些分析类？



3.4.2 确定分析类的职责

- 每一个分析类都有其**职责**，需提供相关的**服务**
- 对象接收的**消息与其承担的**职责****之间存在一一对应关系，即如果一个对象能够接收某项消息，它就应当承担与该消息相对应的职责
- 可用类的**方法名**来表示分析类的职责，并采用简短的自然语言来详细刻画类的职责

示例：确定分析类的职责



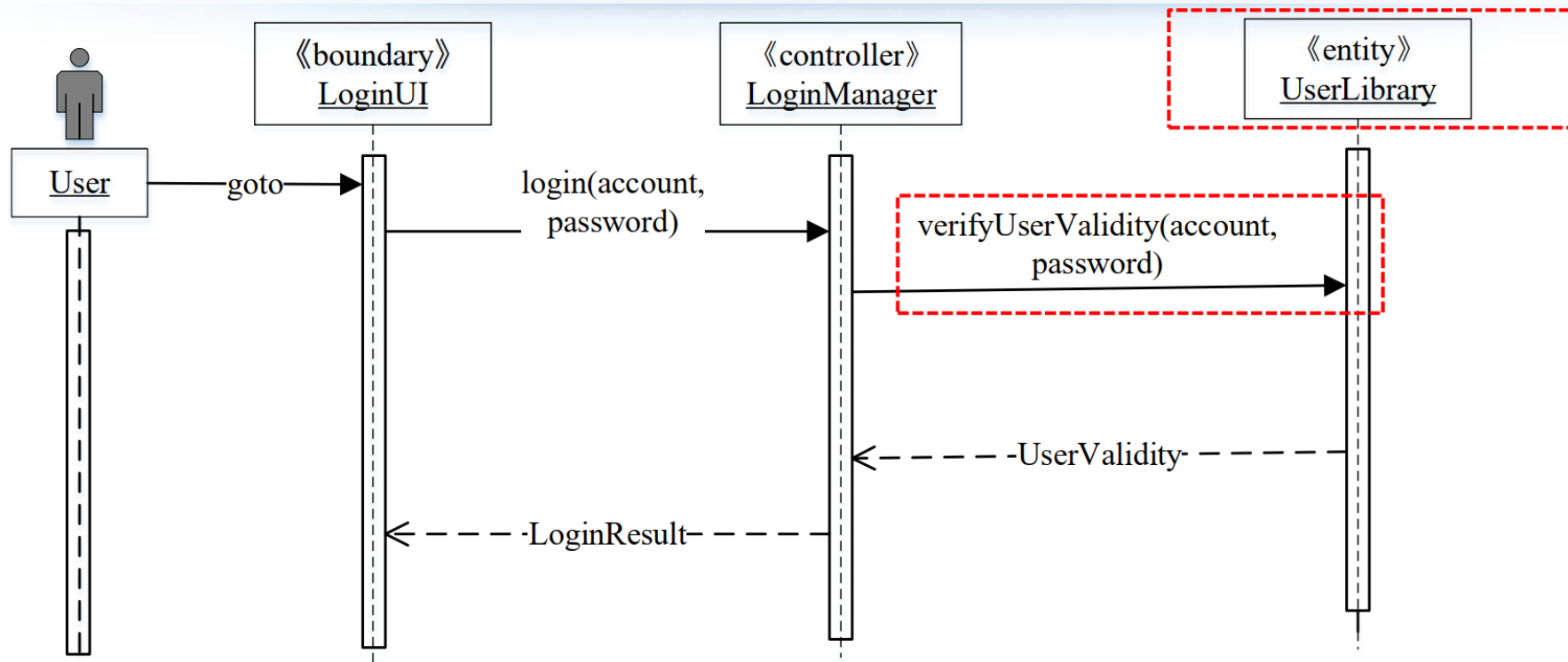
UserLibrary类有何职责？



3.4.3 确定分析类的属性

- 分析类具有哪些属性取决于该类需要持久**保存哪些信息**
- 用例顺序图中的每个对象所发送和接收的消息中往往附带有相关的**参数**，这意味着发送或接收对象所对应的类可能需要**保存和处理与消息参数相对应的信息**，因而可能需要与此相对应的属性

示例：确定分析类的属性



UserLibrary类有何属性?



3.4.4 确定分析类之间的关系

□从全局视角理解和描述不同分析类间的关系

□类之间的关系有多种形式

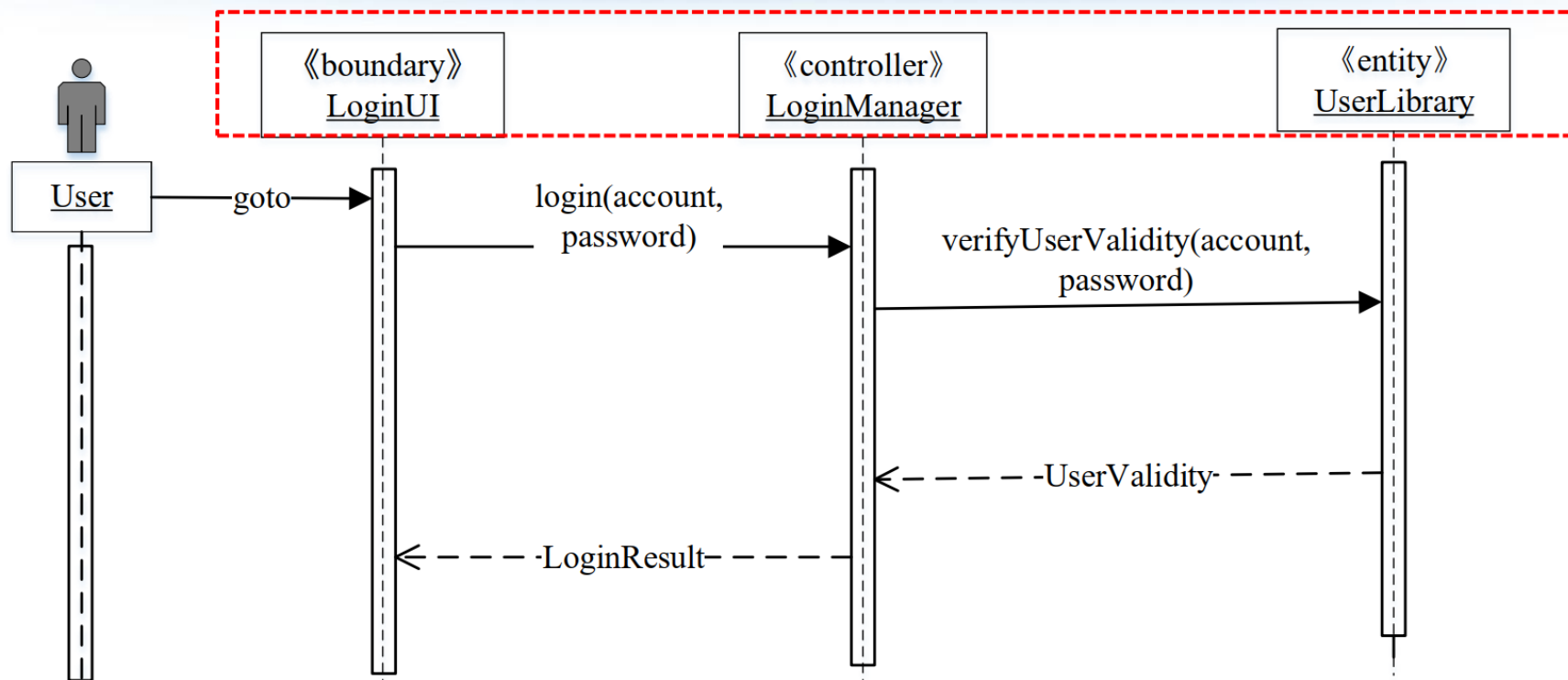
✓包括继承、关联、聚合和组合、依赖等等

□具体策略

✓在用例的顺序图中，如果存在从类A对象到类B对象的消息传递，那么意味着类A和B间存在**关联、依赖、聚合或组合**等关系

✓如果经过上述步骤所得到的若干个类之间存在一般和特殊的关系，那么可对这些分析类进行层次化组织，标识出它们间的**继承关系**

示例：确定分析类之间的关系



哪些分析类之间存在怎样的关系？



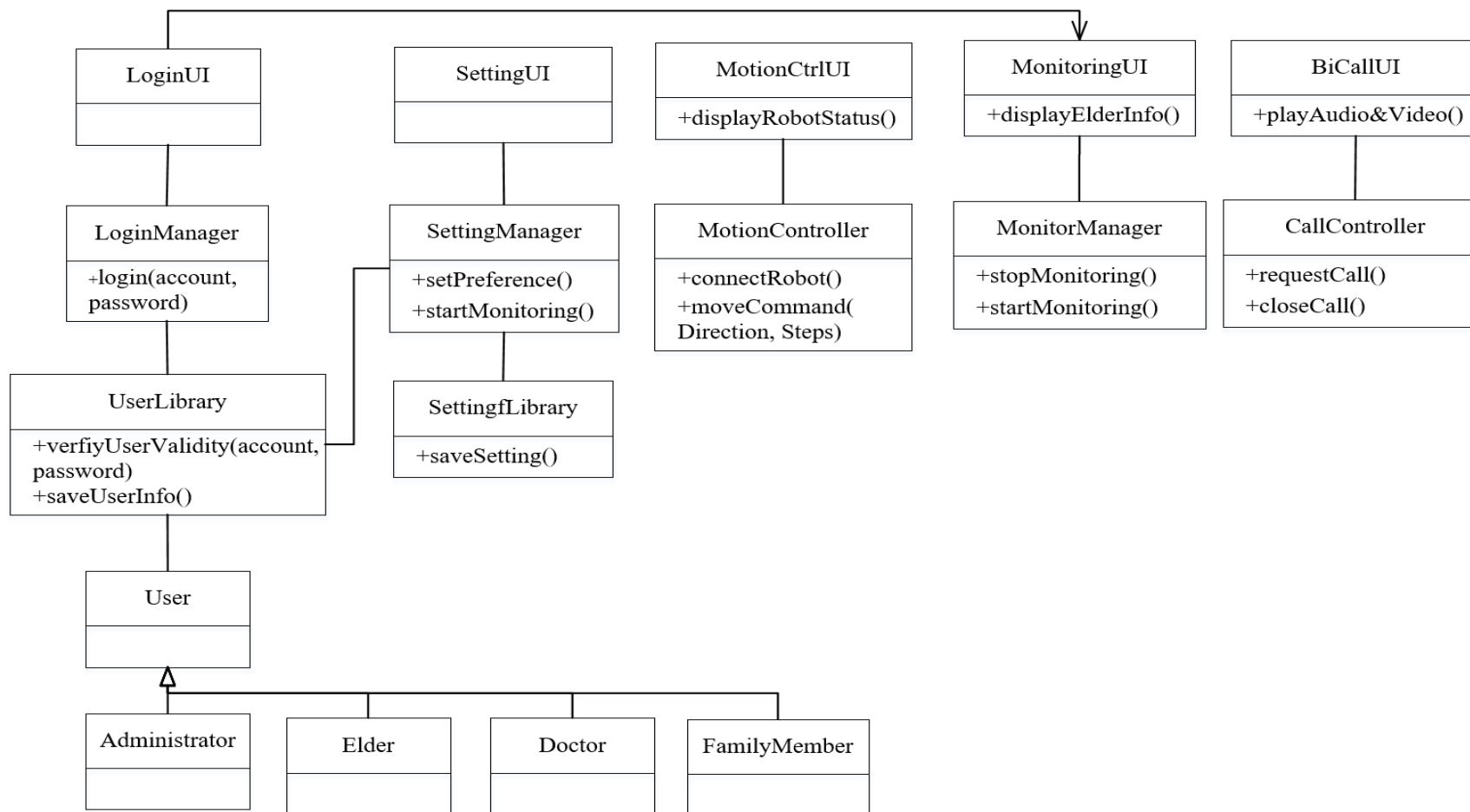
3.4.5 绘制分析类图

□制出系统的分析类图，建立**分析类模型**

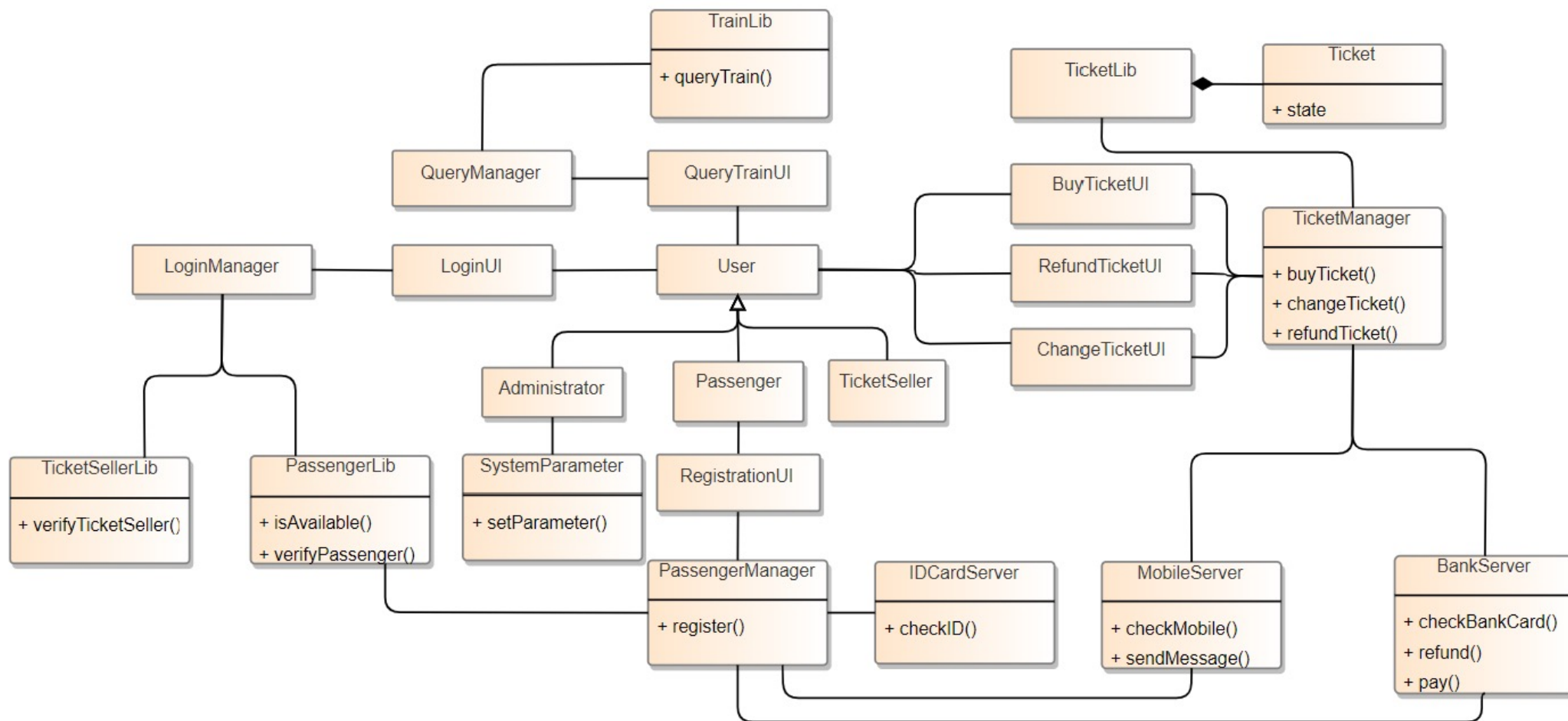
✓直观描述了系统中的**分析类、每个分析类的属性和职责、不同分析类之间的关系**

□如果系列规模较大，分析类的数量多，关系复杂，难以用一张类图来完整和清晰地表示，那么可以分多个子系统来绘制分析类图

示例：“空巢老人看护软件”分析类图



示例：“Mini-12306”分析类图



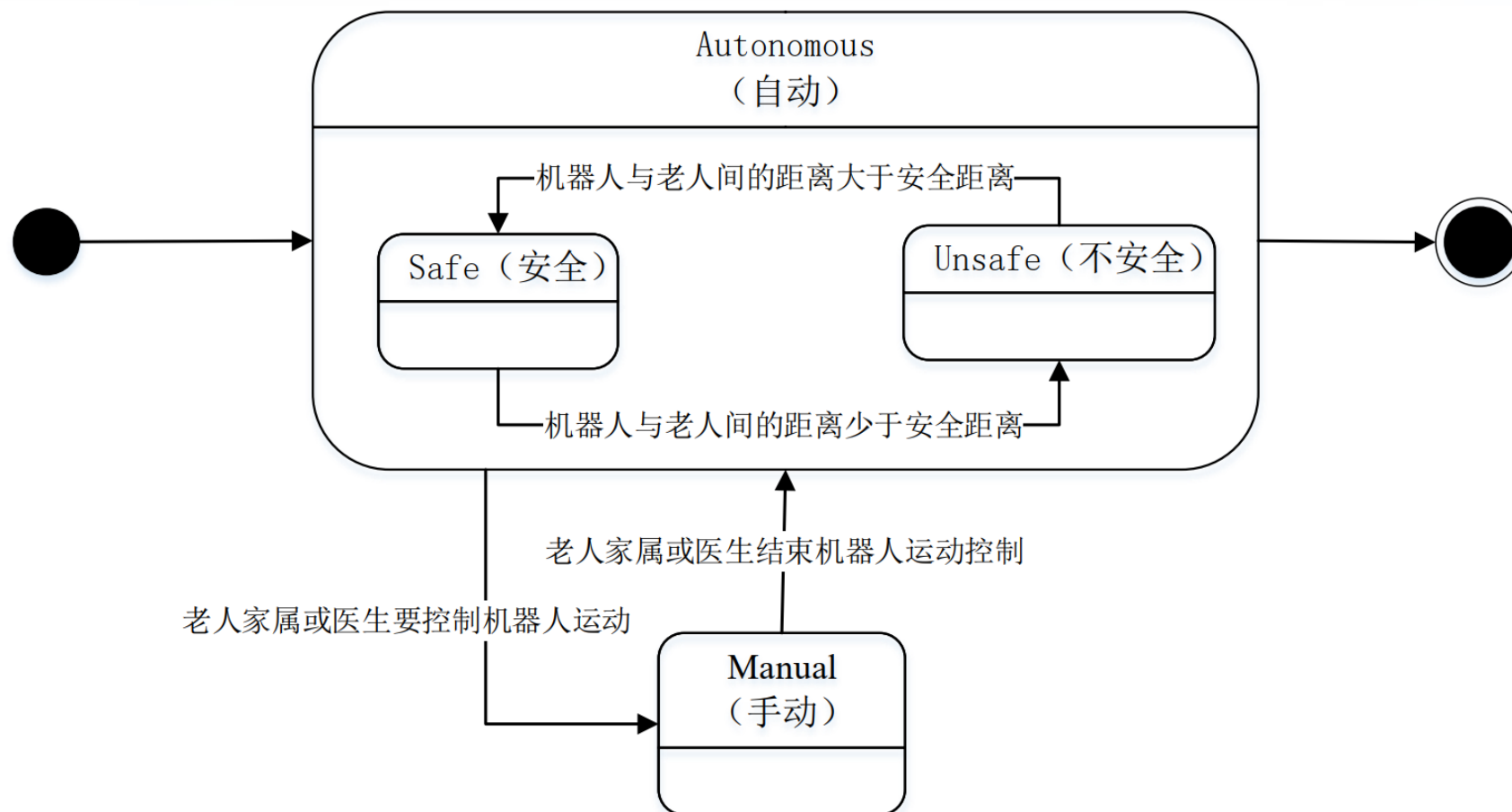
3.5 分析和建立软件需求的状态模型

□用UML的状态图来描述这些对象的**状态模型**，以刻画对象拥有哪些状态、对象的状态如何受事件的影响而发生变化

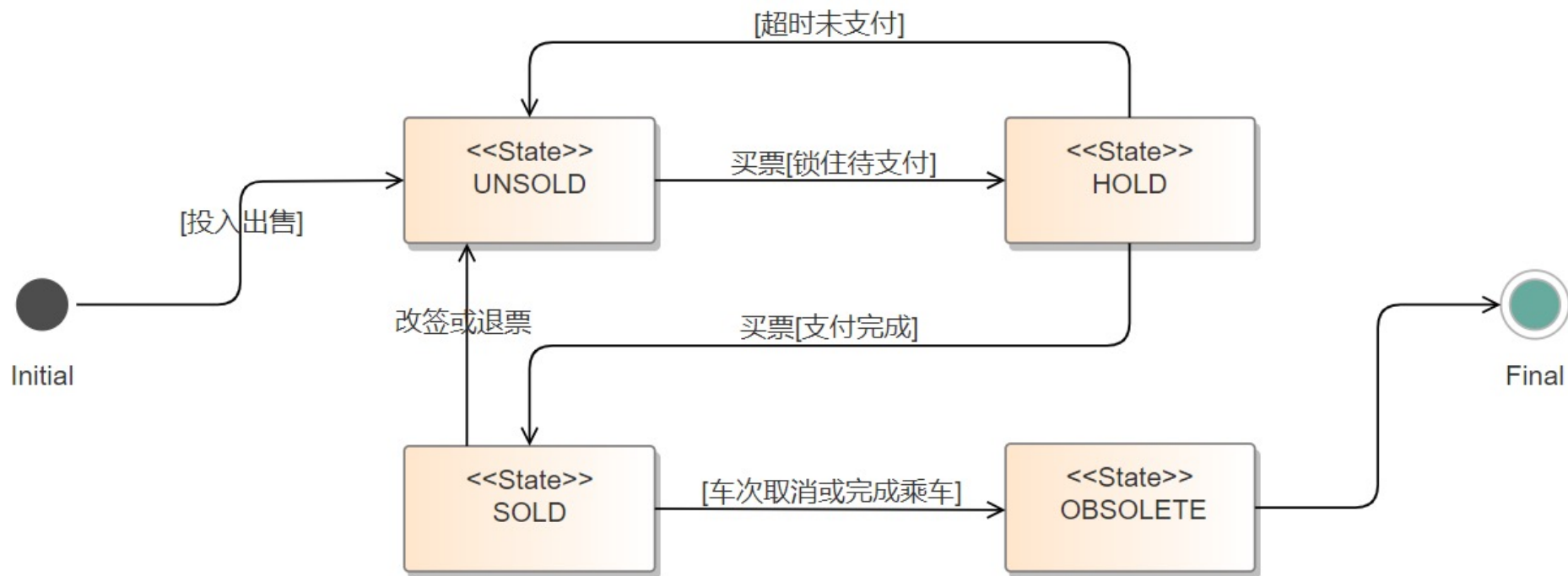
□注意二点

- ✓状态模型是针对**对象**而言的，而非针对分析类
- ✓需求工程师**无需为所有**的类对象建立状态模型，只需针对那些具有复杂状态的对象建立状态模型

示例：“机器人”对象的状态图



示例：“车票”对象的状态图



支持需求分析的CASE工具

□CodeArts Modeling

- ✓由华为研制、支持UML建模的软件工具

□Microsoft Visio

- ✓从严格意义上讲，Microsoft Visio是一个绘图的软件工具

□IBM Rational Rose

- ✓一种面向对象统一建模语言的可视化建模工具

□StarUML

- ✓一款开放源码的UML建模工具

内容

1. 分析软件需求概述

- ✓ 分析软件需求的任务

2. 结构化需求分析方法

- ✓ 过程、建模语言和需求模型

3. 面向对象需求分析方法

- ✓ 过程、建模语言和需求模型

4. 软件需求文档化及评审

- ✓ 软件需求规格说明书及评审



4.1 软件需求文档模板

1. 引言

- 1.1 编写目标
- 1.2 读者对象
- 1.3 文档概述
- 1.4 术语定义
- 1.5 参考文献

2. 软件系统概述

- 2.1 软件产品概述
- 2.2 用户特征
- 2.3 设计和实现约束
- 2.4 假设与依赖

3. 功能性需求描述

- 3.1 软件功能概述
- 3.2 软件需求的用例模型
- 3.3 软件需求的分析模型

4. 非功能性需求

- 5. 界面需求
- 6. 接口定义
- 7. 进度要求
- 8. 交付要求
- 9. 何种形式来交付
- 10. 验收要求

软件需求模型是文档中的重要组成成分

4.2 撰写软件需求文档的注意事项

- **遵循规范**，按照规范来撰写软件需求文档
- **图文并茂**，将软件需求模型以及自然语言描述二者结合在一起，给出软件需求的清晰、准确和详实的表述
- **完整表述**，要给出软件功能性需求和非功能性需求的描述
- **共同参与**，需求工程师要与用户、客户等一起参与软件需求规格说明书的撰写
- **语言简练**，软件需求规格说明书的语言表述要简练，便于阅读和理解
- **前后一致**，在软件需求文档中，对同一个软件需求的表述前后要一致，不要产生相互矛盾或不一致的需求表达

思考和讨论

- 为什么要撰写软件需求文档?
- 没有软件需求文档会产生怎样的情况?



4.3 分析软件需求的输出

□软件原型

- ✓以可运行软件的形式，直观地展示了软件的业务工作流程、操作界面、用户的输入和输出等方面的功能性需求信息

□软件需求模型

- ✓以可视化的图形方式，从多个不同的视角，直观地描述了软件的功能性需求，包括用例模型、用例的交互模型、分析类模型、状态模型等

□软件需求文档

- ✓以图文并茂的方式，结合需求模型以及需求的自然语言描述，详尽刻画了软件需求，包括功能性和非功能性软件需求，软件需求的优先级列表等

4.4 软件需求评审的步骤

- 阅读和汇报软件需求制品
- 收集和整理问题
- 讨论和达成一致
- 纳入配置

为什么要评审软件需求？



评审软件需求 (1/2)

- **内容完整性**，是否包含了用户和客户的所有软件需求
- **内容正确性**，软件需求是否客观、正确地反映了用户和客户的实际期望和要求
- **内容准确性**，是否准确地反映了用户和客户的期望和要求
- **内容一致性**，所描述的软件需求是否存在不一致问题
- **内容多余性**，所描述的软件需求是否都是用户所期望的
- **内容可追踪性**，每一项软件需求是否可追踪的

评审软件需求 (2/2)

- **文档规范性**, 软件需求规格说明书书写是否遵循文档规范
- **图符规范性**, 软件需求模型是否正确地使用了UML的图符
- **表述可读性**, 软件需求文档文字表述是否简洁、可读性好
- **图表一致性**, 软件需求制品中的图表引用是否正确

4.5 如何解决软件需求问题 (1/2)

□ 遗漏的软件需求

- ✓ 再次征求用户、客户、领域专家等意见，以补充遗漏的软件需求

□ 无源头的软件需求

- ✓ 剔除或暂时不用考虑该部分的软件需求，或者将这些软件需求置于低优先级

□ 不一致、相冲突的软件需求

- ✓ 寻找到具有更高级别的用户或客户，由他们来最终确定软件需求

□ 不正确和不准确的软件需求

- ✓ 与用户、客户、领域专家等进行深入的沟通，以正确地理解软件需求的内涵

如何解决软件需求问题 (2/2)

□不规范的软件需求文档

- ✓对照软件需求规范标准和模板，按照其要求来撰写并产生软件需求规格说明书

□不规范和不正确的软件需求模型

- ✓学习UML图符和模型的使用法，在此基础上绘制出正确和规范的UML模型

□费解的软件需求文档

- ✓消除冗余的文字表述，提高语言表达的简洁性，系统梳理和组织软件需求文档的格式，以提高软件需求文档的可读性

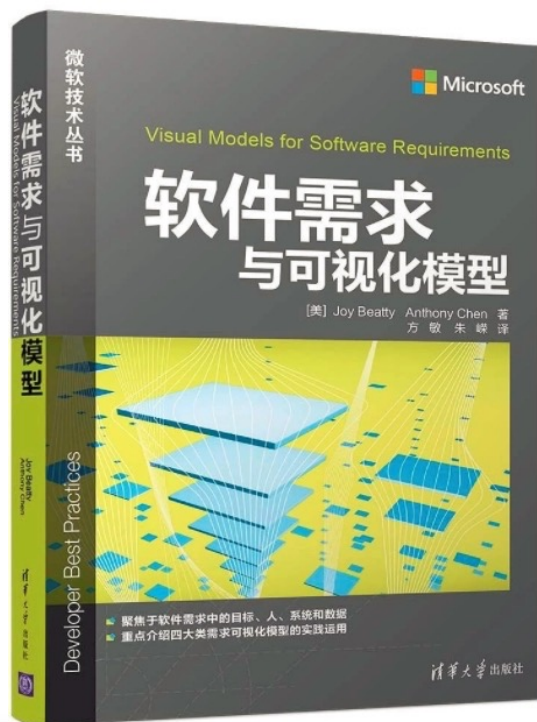
思考和讨论

□如果软件需求模型和文档存在诸多的问题，没有得到有效的解决，会产生什么样的后果？



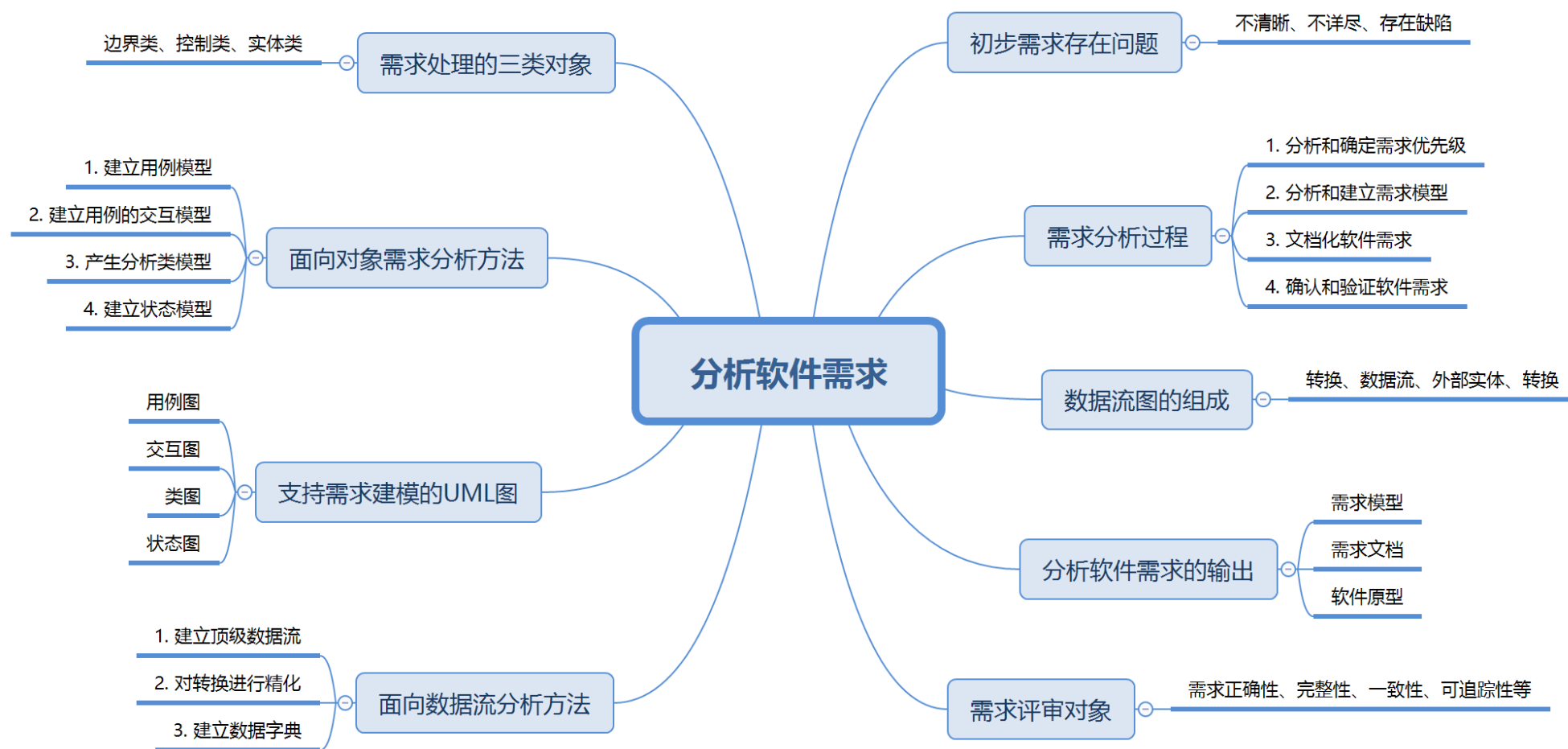
拓展阅读

□ 软件需求与可视化模型, Joy Beatty, Anthony Chen 著, 方敏、朱嵘翻译, 清华大学出版社, 2017



本书侧重于以可视化的方式来表示软件需求, 介绍了四大类二十二个可视化需求模型, 旨在指导读者通过软件需求的可视化模型来进一步明确需求, 促进开发人员和用户等对需求的理解, 推动软件项目的成功开发。本书的内容取自需求领域两位专家十多年的实践经验, 具有重要的指导和参考意义, 可帮助读者准确地描述软件需求, 开发出满足用户需求的软件产品。本书的第一位作者Anthony Chen (安东尼·陈) 是Seilevel联合创始人兼总裁、战略负责人和软件需求创新技术的负责人。

本章知识图谱



小结

□分析软件是要**精化和深化**软件需求

- ✓ 基于初步软件需求，循序渐进
- ✓ 确保软件需求的完整性、一致性和准确性

□**UML**提供了一系列用于描述软件需求的**图**

- ✓ 用例图、交互图、分析类图、状态图

□分析软件需求的**步骤**

- ✓ 分析和确立软件需求优先级、分析和建立软件需求模型、撰写软件需求规格说明书、评审软件需求模型和文档

□分析软件需求的**输出和评审**

- ✓ 软件需求模型、软件需求原型、软件需求文档
- ✓ 评审软件需求制品